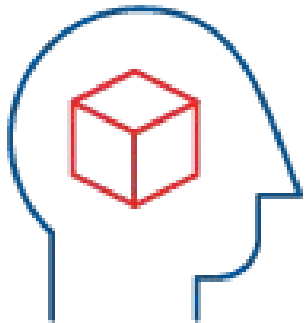




Тестирование мобильных
приложений

Стратегии тестирования



Три основных типа мобильных приложений



Нативное	Создано под конкретную платформу (iOS или Android), использующее её собственные инструменты, а также языки для оптимальной производительности, беспроblemной интеграции, соблюдения стандартов платформы
Кросс-платформенное	Разрабатывается с использованием единого кода, который может работать на нескольких операционных системах (iOS и Android)
Веб-приложение	Работает через браузер на смартфоне, не требует загрузки из маркетплейсов. Доступно как через браузер, так и через иконку на домашнем экране устройства

Для тестировщика понимание разницы между **нативными**, **кросс-платформенными** и **веб-приложениями** критически важно, потому что от этого зависит стратегия тестирования, инструментарий и типичные риски. Ниже — ключевые отличия, которые нужно знать.

Три основных типа мобильных приложений



1. Нативные приложения

Создаются под конкретную ОС (iOS — Swift/Objective-C, Android — Kotlin/Java) с использованием родных SDK. Что важно для тестировщика:

1. Установка и обновления — через магазины (App Store, Google Play). Требуется проверка подписи, апдейтов с сохранением данных.
2. Доступ к железу — полный доступ ко всем датчикам, камере, NFC, биометрии. Тестирование этих функций обязательно на реальных устройствах.
3. Производительность — высокая, но нужно тестировать потребление батареи, памяти, холодный старт.
4. UI/UX — строго соответствует гайдлайнам платформы (Material Design для Android, Human Interface Guidelines для iOS). Разные навигационные паттерны, жесты.
5. Тестирование прерываний — критически важно (звонок, сворачивание, режим энергосбережения).
6. Инструменты — ADB, Xcode, логи, эмуляторы/симуляторы, профилировщики (Instruments, Android Profiler).
7. Автоматизация — Espresso (Android), XCUITest (iOS), Appium (кросс-платформенно, но с учётом нативных элементов).

Три основных типа мобильных приложений



2. Кросс-платформенные приложения

Написаны на одном языке (JavaScript, Dart, C#) и компилируются в нативный код под обе ОС. Популярные фреймворки: React Native, Flutter, Xamarin.

Что важно для тестировщика:

- 1. Единая кодовая база** — баг может проявиться только на одной платформе из-за особенностей рендеринга или мостов (bridges).
- 2. UI-слои:**
 - **React Native** — использует нативные компоненты (через мост), поэтому поведение UI близко к нативному, но есть риск рассинхрона.
 - **Flutter** — всё рисуется движком Skia, поэтому UI идентичен на обеих платформах, но могут быть расхождения в нативных наложениях (например, клавиатура, системные диалоги).
- 3. Производительность** — часто ниже нативной, особенно при сложной анимации или тяжёлых расчётах. Нужно тестировать плавность скролла, время запуска, потребление памяти (утечки в JavaScript-движке).
- 4. Доступ к железу** — через плагины/мосты. Ограничения: некоторые функции могут быть не реализованы или работать некорректно на одной из платформ.
- 5. Обновления** — возможны «горячие» обновления (over-the-air) без публикации в магазине (например, CodePush для React Native). Это удобно, но несёт риски для безопасности и целостности данных.
- 6. Инструменты** — помимо нативных инструментов, добавляются специфические:
 - React Native: react-devtools, Flipper, логи из JS-стороны.
 - Flutter: DevTools, виджет-инспектор.
- 7. Совместимость** — нужно тестировать на разных версиях ОС, так как мосты могут вести себя по-разному.

Три основных типа мобильных приложений



3. Веб-приложения (Mobile Web)

Это обычные сайты, адаптированные под мобильные браузеры. Работают в браузере (Safari, Chrome) и не требуют установки.

Что важно для тестировщика:

1. Отсутствие установки — нет проверки через магазины. Доступны по URL.
2. Ограниченный доступ к железу — только через HTML5 API (геолокация, акселерометр, камера — через getUserMedia). Многие функции недоступны.
3. Производительность — зависит от браузера, сети и мощности устройства. Критически важно тестировать на медленных сетях и слабых устройствах.
4. Кросс-браузерность — поведение может различаться в Chrome (Android) и Safari (iOS), а также в разных версиях. Особенно это касается поддержки CSS-свойств, JavaScript API, localStorage.
5. Офлайн-режим — ограничен работой Service Workers. Нужно тестировать, как приложение ведёт себя без интернета.
6. Инструменты — Chrome DevTools, Safari Web Inspector, эмуляция устройств в браузере, прокси (Charles, Fiddler) для подмены ответов.
7. Особенности iOS — Safari может кэшировать ответы агрессивно, есть ограничения на выполнение JavaScript в фоне, а также политика обновления Service Workers.

Три основных типа мобильных приложений



Сравнительная таблица для тестировщика

Параметр	Нативное	Кросс-платформенное	Веб-приложение
Установка	Через магазин, APK/IPA	Через магазин	По URL, нет установки
Доступ к железу	Полный	Через плагины, может быть ограничен	Ограничен HTML5 API
Производительность	Оптимальная	Средняя (зависит от фреймворка)	Низкая (зависит от браузера)
UI/UX	Платформенный (HIG/Material)	Единый или гибридный	Единый для всех (адаптив)
Обновления	Только через магазин	Через магазин + возможно OTA	На стороне сервера (мгновенно)
Тестирование прерываний	Критично (фон, звонки)	Критично (фон может сбрасывать состояние)	Менее актуально (браузер управляет)
Офлайн-режим	Полноценный (хранилище, БД)	Полноценный (через нативные механизмы)	Ограничен (Service Workers, кэш)
Безопасность	Высокая (песочница, подпись)	Средняя (зависит от фреймворка)	Низкая (XSS, CSRF, подмена)
Автоматизация	Espresso, XCUITest, Appium	Appium, Flutter Driver, Detox	Selenium, Puppeteer, Playwright
Отладка	ADB, Xcode, логи	Flipper, React/Flutter DevTools	DevTools браузера

На чем тестировать Android-приложения:



Для тестирования Android-приложений существует несколько подходов, и у каждого есть свои сильные стороны и ограничения. Начинающему тестировщику важно понимать, **когда и что** выбирать.

Эмулятор

- ✓ Бесплатно
- ✓ Быстро
- ✓ Любая версия Android в наличии

- ✗ Не отражает реальную производительность

Реальное устройство

- ✓ Точно
- ✓ Аппаратура
- ✓ Сеть
- ✓ Прерывания

- ✗ Дорого
- ✗ Сложно масштабировать

Облачные фермы

- ✓ Доступ
- ✓ Тысячи устройств
- ✓ CI/CD готово

- ✗ Задержка
- ✗ Очереди
- ✗ Стоимость при интенсивном использовании

Гибридная стратегия

Ранние этапы → Эмулятор

Основное тестирование → Реальные устройства

Совместимость → Облачная ферма

Три основных типа мобильных приложений

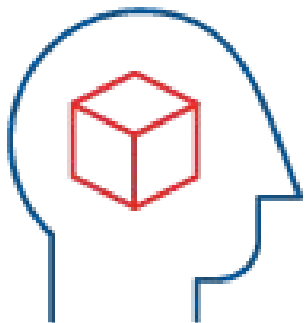


Наиболее общие рекомендации для тестировщика

- 1. Не делайте предположений** — поведение одинакового на первый взгляд функционала может кардинально отличаться на iOS и Android даже в кросс-платформенном приложении.
- 2. Всегда проверяйте на реальных устройствах** — особенно аппаратные функции, производительность и прерывания. Для веб-приложений — на разных браузерах.
- 3. Учитывайте жизненный цикл:**
 1. Нативные: тестируйте установку/обновление, фоновую активность, push-уведомления.
 2. Кросс-платформенные: проверяйте стабильность мостов, утечки памяти в JS-слое.
 3. Веб: тестируйте на медленных сетях, в режиме инкогнито (без кэша), с отключёнными Service Workers.
- 4. Выбирайте инструменты под тип** — для веб-приложения не нужно ADB, а для нативного — обязательно.
- 5. Знайте ограничения платформ** — например, на iOS нельзя эмулировать низкий заряд батареи в симуляторе, на Android — доступ к некоторым системным настройкам может быть ограничен без root. Эти отличия помогут правильно строить стратегию тестирования и не пропустить специфические для каждого типа приложения дефекты.

Тестирование мобильных приложений

Что тестируем?



Чек-лист тестирования мобильных приложений

Чек-лист для тестирования мобильных приложений состоит из восьми разделов:

1. Функциональное тестирование
2. Тестирование совместимости
3. Тестирование безопасности
4. Тестирование локализации и глобализации
5. Тестирование удобства использования
6. Стрессовое тестирование
7. Кросс-платформенное тестирование
8. Тестирование производительности

Чек-лист тестирования мобильных приложений

Функциональное тестирование



В данном пункте нам важно убедиться, что наш продукт соответствует нужной функциональной спецификации, упомянутой в документации по разработке.

Что проверяем?

1. Установка/удаление/накатка версий
2. Запуск приложения (отображение Splash Screen)
3. Работоспособность основного функционала приложения
 - 3.1 Авторизация (по номеру телефона/через соц. сети/e-mail)
 - 3.2 Регистрация (по номеру телефона/через соц. сети/e-mail)
 - 3.3 Онбординг новых пользователей
 - 3.4 Валидация обязательных полей
 - 3.5 Навигация между разделами приложения
 - 3.6 Редактирование данных в профиле пользователя
 - 3.7 Проверка оплаты
 - 3.8 Тестирование фильтров
 - 3.9 Бонусы

Чек-лист тестирования мобильных приложений

Функциональное тестирование (продолжение)



Что проверяем?

4. Корректное отображение ошибок.
5. Работа с файлами (отправка/получение/просмотр).
6. Тестирование тайм-аутов.
7. Тестирование заглушек (нет соединения с интернетом/нет, например, товаров и т.д).
8. Тестирование pop-up, алертов.
9. Тестирование WebView.
10. Скролл/свайп элементов.
11. Тестирование PUSH уведомлений.
12. Сворачивание/разворачивание приложения.
13. Разные типы подключений (сотовая связь/Wi-Fi).
14. Ориентация экрана (альбомная/портретная).
15. Темная/светлая темы.
16. Реклама в приложении.
17. Шаринг контента в соц. Сети.
18. Работа приложения в фоне.
19. Пагинация страниц.
20. Политики конфиденциальности и прочие ссылки на документы.

Чек-лист тестирования мобильных приложений

Функциональное тестирование (пояснения)



Мобильные девайсы устроены так, что приложение может быть как в активном режиме, так и в бэкграунде. Например, если мы переключились на другое приложение или у нас входящий звонок.

Предположим, что по какой-то причине наш пользователь перевел приложение в бэкграунд. А затем вернулся.

Вот несколько примеров того, что может быть не так. Например, у нас приложение-чат и вся история переписки потерялась. Теперь требуется перезагрузка страницы. Плохо? Конечно! Или у нас приложение “записная книжка”. Пользователь не успел что-то дописать, когда его прервал входящий звонок. Вернувшись, он обнаружил, что написанное им стерлось. Теперь придется писать все заново. А может лучше не пользоваться таким приложением?

Обязательно проверяйте, как ключевой функционал приложения работает после выхода из бэкграунда.

Чек-лист тестирования мобильных приложений

Функциональное тестирование (пояснения)



Электронная клавиатура

Чаще всего пользователи Mobile Web используют электронную клавиатуру для ввода текста. Бывает такое, что при ее появлении верстка приложения ломается. Обычно эта проблема связана с тем, как рассчитываются пропорции экрана и как они влияют на эту самую верстку.

Стоит обратить особое внимание на те страницы приложения, где пользователю приходится заполнять много данных: страница регистрации, какие-нибудь опросники и так далее.

Ориентация экрана

У мобильных приложений есть два типа ориентации экрана: portrait и landscape.

Если мы “положим на бок” наш девайс, большая часть мобильных приложений “перерисовется” под новые пропорции экрана. Это касается и мобильных браузеров. В этот момент верстка нашего веб-приложения также может сломаться в самых непредсказуемых местах.

Chrome DevTools умеет эмулировать оба режима, но все же сам процесс переворота и дальнейшая перерисовка веб-приложения происходит иначе, чем на настоящем мобильном браузере.

То, как наше приложение будет выглядеть после перерисовки обязательно стоит проверить. Причем, желательно, в обе стороны: из portrait в landscape и из landscape в portrait.



Обработка тапов

В отличие от Desktop Web, где есть только клик мышью, в мобильном приложении существует несколько способов взаимодействия с интерфейсом: touch, tap, flick и так далее.

Chrome DevTools умеет эмулировать часть этих действий. Но, во-первых, не все. А, во-вторых, не всегда результат во время эмуляции такой же, как во время использования реального девайса.

Даже если ваше мобильное веб-приложение не заточено под специальные действия, все равно стоит проверить взаимодействие интерфейса хотя бы с tap'ом. Особенно такие места, как меню или какие-нибудь переключатели. Суть в том, что tap не попадает в какие-то конкретные координаты. Он попадает на целую область. Если ваши элементы управления находятся рядом, tap может влиять на несколько элементов сразу и доставлять неудобство пользователю.

Тестирование совместимости



Тестирование совместимости используется, чтобы убедиться, что ваше приложение совместимо с другими версиями ОС, различными оболочками и сторонними сервисами, а также аппаратным обеспечением устройства.

Что проверяем?

1. Корректное отображение гео-позиции.
2. Информацию об операциях (чеки и т.д.).
3. Различные способы оплаты (Google Pay, Apple Pay).
4. Тестирование датчиков (освещенности, температуры устройства, гироскоп и т.д.).
5. Тестирование прерываний (входящий звонок/смс/push/будильник/режим «Не беспокоить» и т.д.).
6. Подключение внешних устройств (карта памяти/наушники и т.д.).



Тестирование безопасности

Данная проверка нацелена на поиск недостатков и пробелов с точки зрения безопасности приложения.

Что проверяем?

1. Тестирование разрешений (доступ к камере/микрофону/галерее/и т.д.).
2. Данные пользователя (пароли) не передаются в открытом виде.
3. В полях, с вводом пароля и подтверждением пароля, данные скрываются астерисками.

Тестирование локализации и глобализации



Тестирование интернационализации/глобализации приложения включает тестирование приложения для различных местоположений, форматов дат, чисел и валют, а также замену фактических строк псевдостроками. Тестирование локализации включает тестирование приложения с локализованными строками, изображениями и рабочими процессами для определенного региона.

Что проверяем?

1. Все элементы в приложении переведены на соответствующий язык.
2. Тексты защиты внутри приложения и пользователь в настройках приложения может выставить необходимый язык.
3. Тексты зависят от языка в системных настройках.
4. Тексты приходят с сервера.
5. Корректное отображение форматов дат (ГОД — МЕСЯЦ — ДЕНЬ или ДЕНЬ — МЕСЯЦ — ГОД.).
6. Корректное отображение времени в зависимости от часового пояса.

Чек-лист тестирования мобильных приложений



Тестирование удобства использования



Тестирование удобства использования помогает удостовериться в простоте и эффективности использования продукта пользователем, с целью достижения поставленных целей. Иными словами, это не что иное, как тестирование дружелюбности приложения для пользователя.

Что проверяем?

1. Корректное отображение элементов на устройствах с различными разрешениями экранов.
2. Все шрифты соответствуют требованиям.
3. Все тексты правильно выровнены.
4. Все сообщения об ошибках верные, без орфографических и грамматических ошибок.
5. Корректные заголовки экранов.
6. В поисковых строках присутствуют плейсхолдеры.
7. Неактивные элементы отображаются серым.
8. Ссылки на документы ведут на соответствующий раздел на сайте.
9. Анимация между переходами.
10. Корректный возврат на предыдущий экран.
11. Поддерживаются основные жесты при работе с сенсорными экранами (swipe back и т.д.).
12. Пиксель-перфект (точное соответствие макету дизайна).

Стрессовое тестирование



Стрессовое тестирование направлено на определение эффективности производительности приложения в условиях повышенной нагрузки. Стресс-тест в этом контексте ориентирован только на мобильные устройства.

Что проверяем?

1. Высокая загрузка центрального процессора.
2. Нехватка памяти.
3. Загрузка батареи.
4. Отказы.
5. Низкая пропускная способность сети.
6. Большое количество взаимодействий пользователя с приложением (для этого может понадобиться имитация реальных условий состояния сети).

Чек-лист тестирования мобильных приложений

Кросс-платформенное тестирование



Важный вид тестирования, который необходимо проводить для понимания того, будет ли должным образом отображаться тестируемый продукт на различных платформах, используемых целевой аудиторией.

Что проверяем?

— Работоспособность приложения на различных устройствах разных производителей (**об использовании эмуляторов далее**).



Тестирование производительности

Если пользователь устанавливает приложение, и оно не отображается достаточно быстро (например, в течение трех секунд), оно может быть удалено в пользу другого приложения. Аспекты потребления времени и ресурсов являются важными факторами успеха для приложения, и для измерения этих аспектов проводится тестирование производительности.

Что проверяем?

1. Время загрузки приложения.
2. Обработка запросов.
3. Кэширование данных.
4. Потребление ресурсов приложением (например расход заряда батареи).

Дополнительные важные сценарии: батарея, режим ожидания, блок экрана, сеть



Дополнительные сценарии проверки приложения (железо).

Батарея

Сценарии, связанные с батареей:

1. Проверка, не расходует ли приложение батарею слишком активно, это одна из первых причин удаления приложений, и на это следует обращать особое внимание. Если это не навороченная игра с графикой, приложение должно потреблять по возможности минимум ресурса батареи, к этому следует стремиться.
2. Также проверьте расход батареи при работе приложения в фоновом режиме. Оно также должно быть по возможности минимальным.

Режим простоя

Сценарии, связанные со спящим режимом девайса:

1. Если в приложении реализована функция входа в систему, можно проверить, не происходит ли выход (вылогинивание) пользователя из приложения после того, как оно перешло в фоновый режим.
2. Если у вас приложение, связанное с платежами, и например обрабатывающее платежи в фоновом режиме, то приложение должно работать без сбоев и успешно выполнять требуемые транзакции.
3. Если телефон в спящем режиме, приложение не должно сильно потреблять ресурс батареи; также можно проверить, не нагревается ли телефон в спящем режиме из-за фоновой работы приложения, это частая жалоба пользователей в сторе.
4. Что происходит если пользователь держит приложение в фоне и через какое-то время пытается открыть с того места, где он его оставил (частое «место поломки»). Это чисто технический сценарий, который лучше подробно обсудить с командой.

Дополнительные важные сценарии: батарея, режим ожидания, блок экрана, сеть



Блок экрана

1. Также проверим сценарии с блокировкой экрана; например, приложением запрошено получение данных, и в это время телефон пользователя блокируется; тогда приложение должно оставаться активным в фоновом режиме и успешно завершить нужную операцию.
2. Другой сценарий — если пользователь намеренно блокирует телефон во время выполнения операции, и в этом случае также нужно проверить, что операция будет успешно завершена и не будет отменена.
3. Еще один сценарий — если телефон заблокирован во время работы с приложением, проверить расход заряда батареи.

Сеть

Сценарии, связанные с подключением к Интернету/WiFi и сервисами определения местоположения:

1. Проверить, не нарушается ли работа приложения при выполнении какой-либо операции, когда интернет на телефоне отключается. В этом случае пользователю должен быть показано соответствующее сообщение.
2. Далее проверить, когда интернет появится и пользователь снова в сети, в этом случае приложение должно работать корректно и сообщение «Нет интернета» должно исчезнуть и ошибок не должно возникать.

Дополнительные важные сценарии: батарея, режим ожидания, блок экрана,  сеть



Сеть

3. Пользователь должен иметь возможность бесппроблемно пользоваться приложением как при подключении к WiFi, так и к мобильному Интернету.
4. Другим способом проверки является подключение телефона к мобильному хот-споту и проверка работоспособности приложения; хотя этот сценарий не требует длительного глубокого тестирования, однако провести такой тест будет полезно.
5. Проверка режима В Самолете, поведение приложения. Если приложение имеет какую-либо зависимость, или например это приложение, связанное со смарт-часами, то устройство должно переходить в режим без ошибок, и это должно происходить плавно.
6. Аналогично, при отключении этого режима все должно вернуться в нормальное состояние при минимальном вмешательстве пользователя.
7. Включение/выключение геолокации в телефоне и проверка работы с геолокацией, соответствующие сообщения и пр.

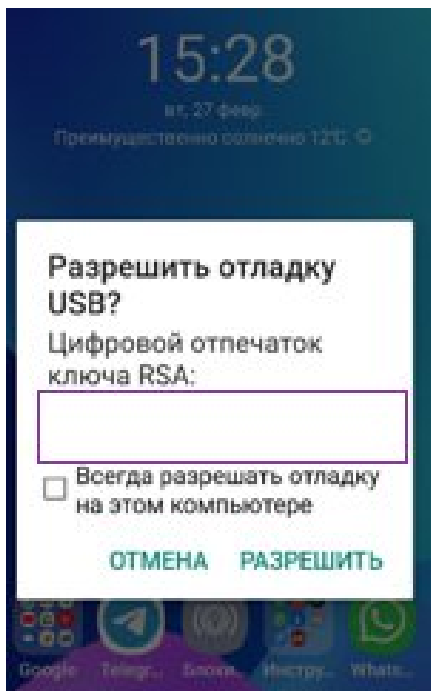
Проверяем, по мере возможности

Инструменты тестирования мобильных пр

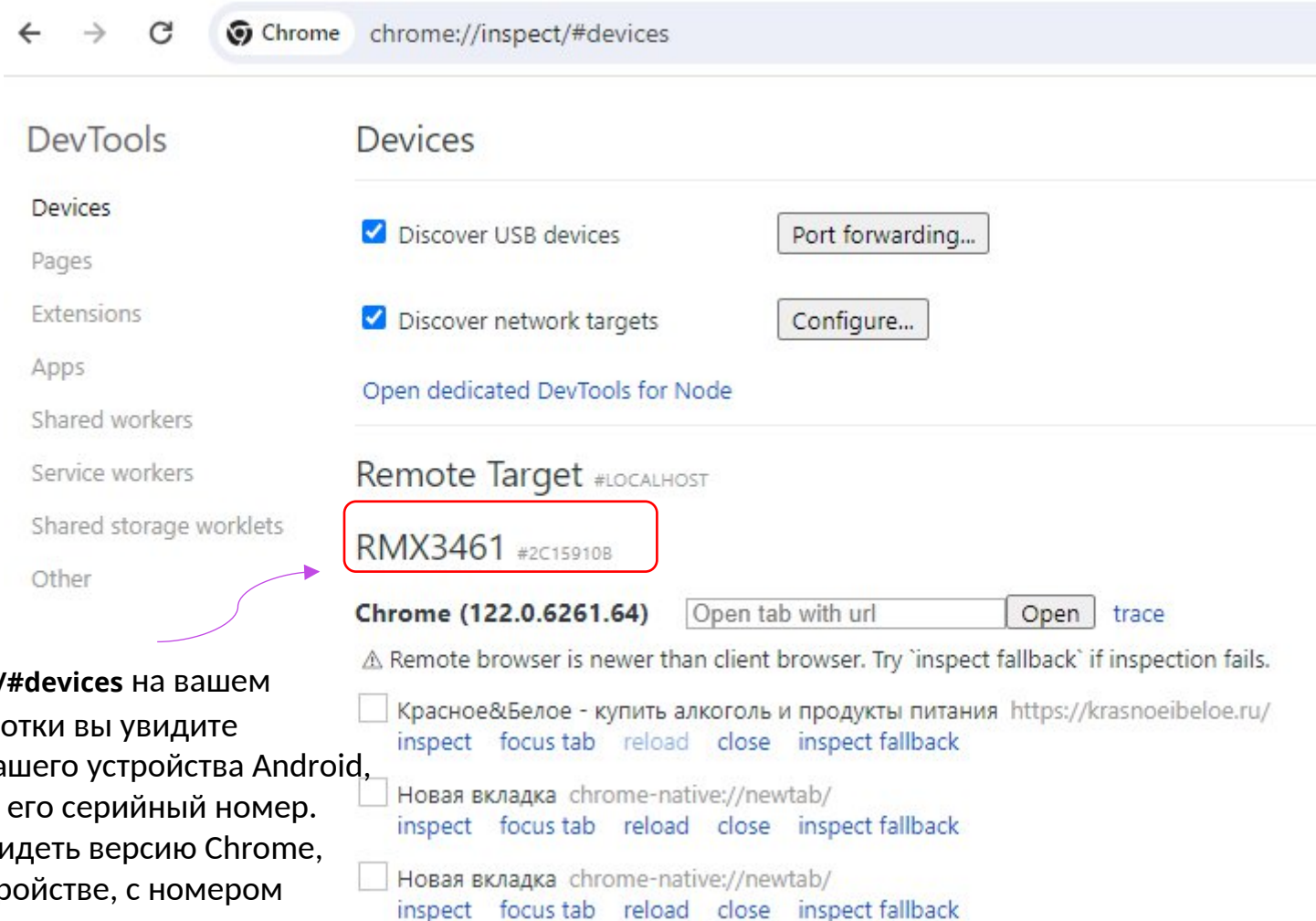
Тестирование мобильного веб-приложения в DevTools

Подключить устройство к компьютеру и 1)разрешить отладку по USB, 2) Зайти на компьютере в `chrome://inspect#devices`, откройте Chrome на своем устройстве Android.

1)



2)



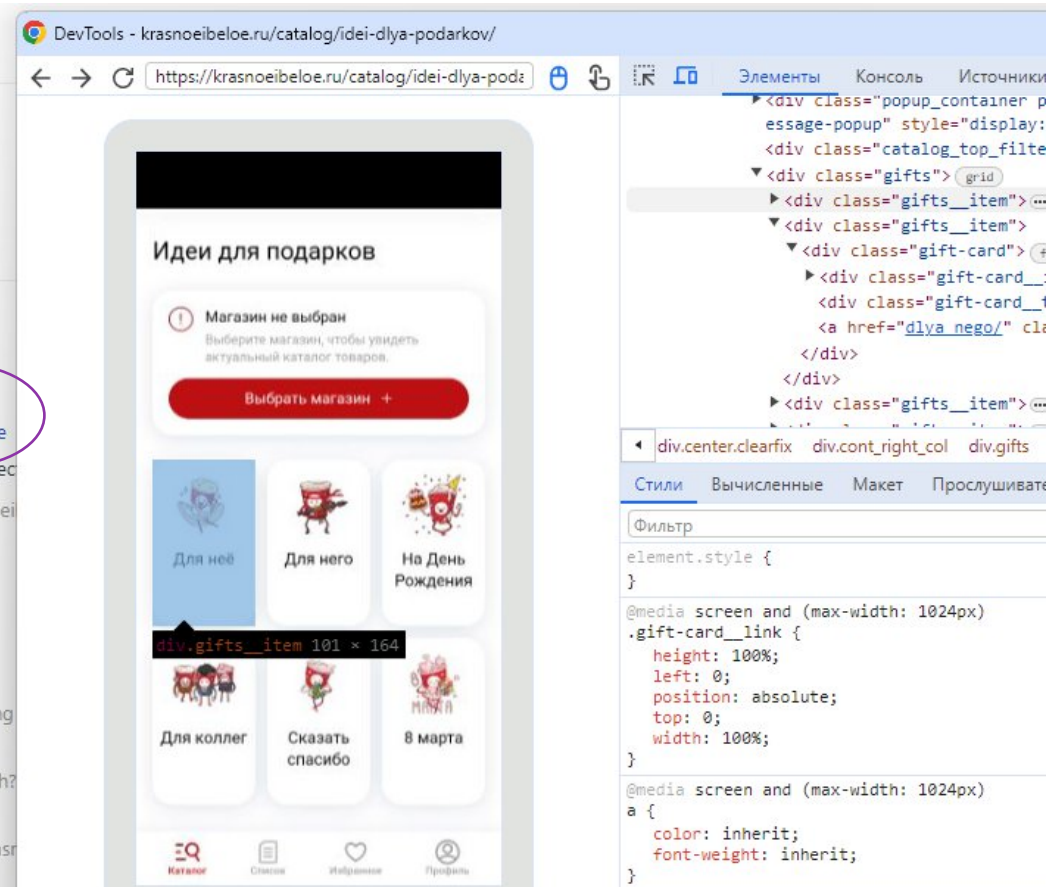
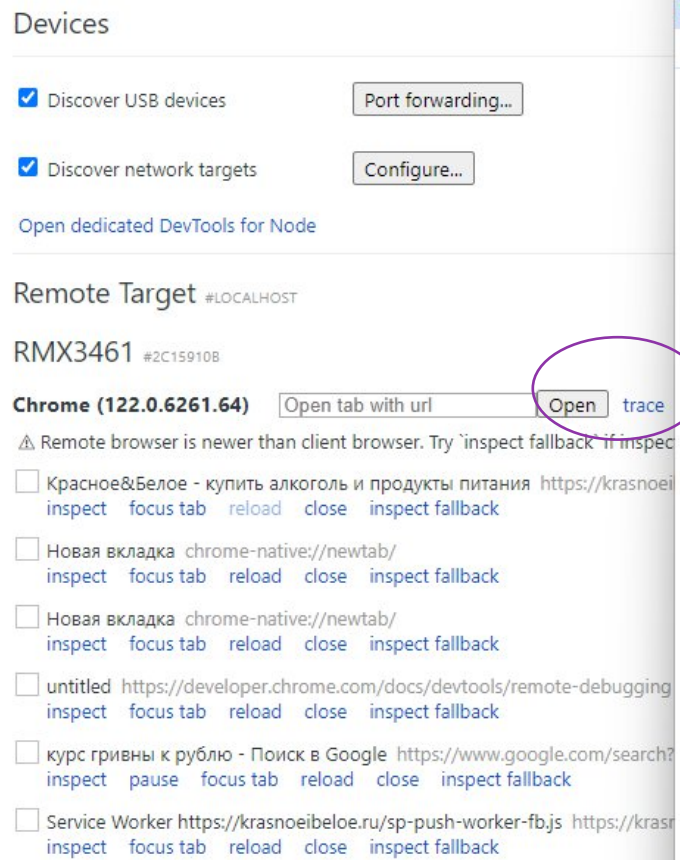
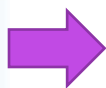
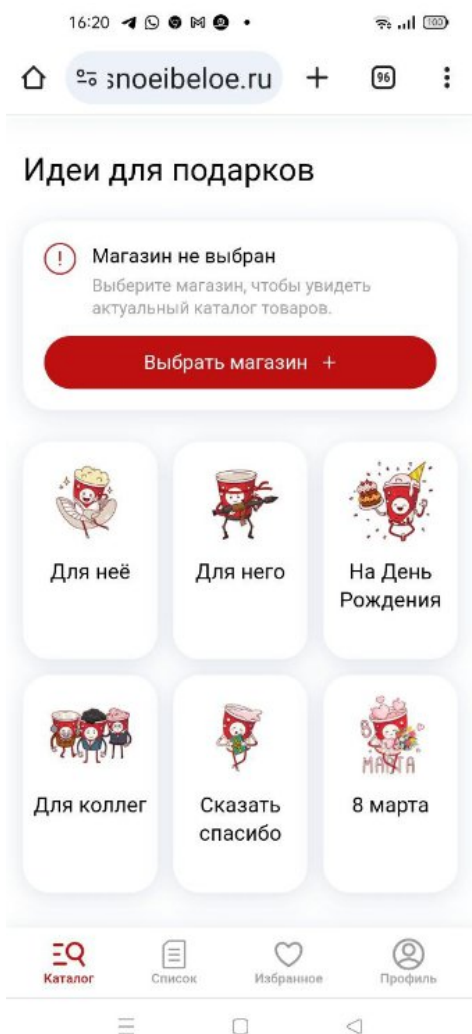
В `chrome://inspect#devices` на вашем компьютере разработки вы увидите название модели вашего устройства Android, за которым следует его серийный номер. Ниже вы можете увидеть версию Chrome, работающую на устройстве, с номером версии в скобках.

Тестирование мобильного приложения в DevTools

Зайти в браузере компьютера на выбранную страницу



3)



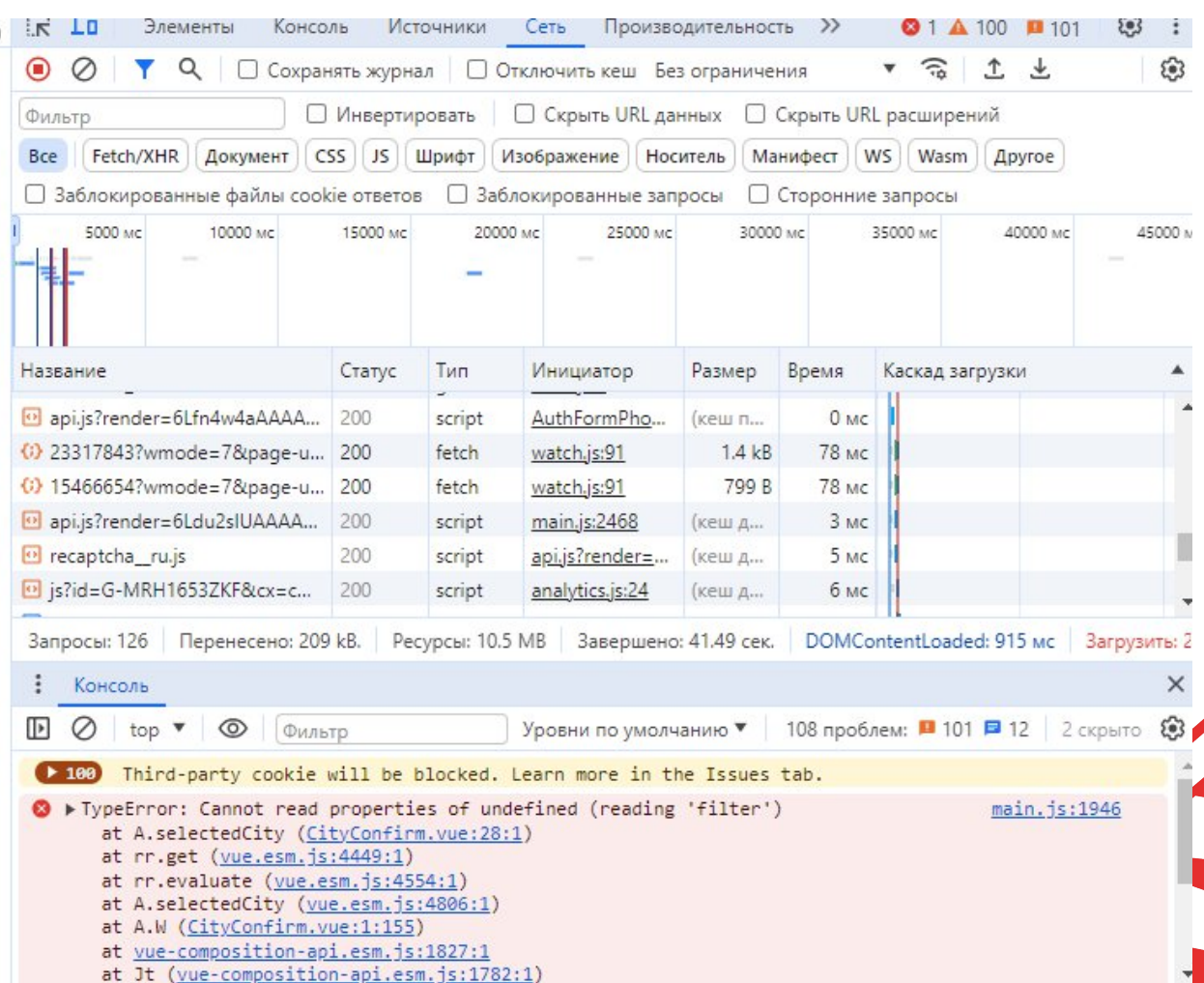
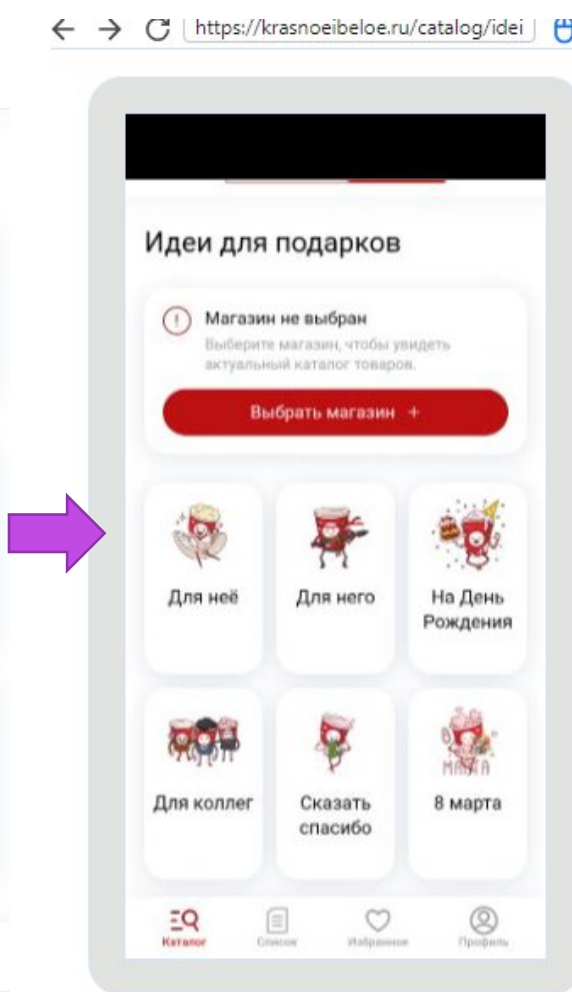
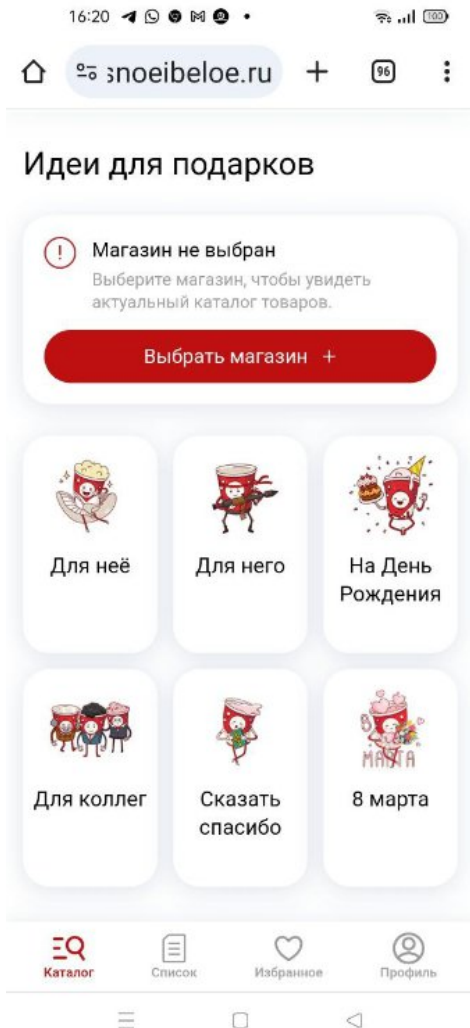
Подробнее о подключении к DevTools здесь:

<https://developer.chrome.com/docs/devtools/remote-debugging?hl=ru#troubleshooting>

Тестирование мобильного веб-приложения в DevTools

Сетевые возможности мы тоже сможем оценить

4)



Тестирование мобильного веб-приложения в DevTools

Этот метод подходит **не для всех типов приложений**. Он предназначен исключительно для отладки **веб-содержимого**, а не нативного кода.

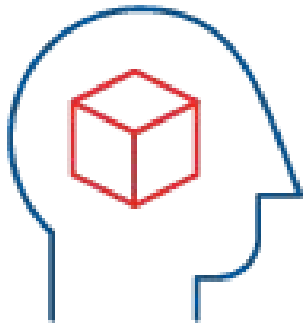
Вот краткая таблица совместимости, чтобы вы могли быстро ориентироваться:

Тип приложения	Можно ли тестировать через chrome://inspect?	Что именно можно тестировать?
Нативное приложение (Native)	Нет	Нельзя. Этот метод не показывает нативный UI (кнопки, списки, экраны, написанные на Kotlin/Java) и не дает доступа к нативному коду.
Кросс-платформенное приложение (React Native, Flutter, Xamarin)	Да, с оговорками	Можно тестировать только веб-слои. Для React Native это отладка JavaScript-кода и стилей. Flutter имеет свой собственный механизм отладки (DevTools).
Гибридное приложение (Hybrid)	Да	Можно тестировать веб-части (HTML, CSS, JS), которые загружаются в компонент WebView.
Мобильное веб-приложение (Mobile Web)	Да	Можно полноценно тестировать сайт, открытый в браузере на устройстве. Это и есть основной сценарий для этого метода.

Кросс-платформенные приложения (React Native, Flutter) – самая интересная ситуация

React Native использует "мост" для преобразования JS-кода в нативные компоненты. Хотя вы не увидите сами нативные компоненты в chrome://inspect, сам метод часто используется разработчиками для отладки **JavaScript-логики** React Native-приложений. Flutter же рендерит все самостоятельно (через Skia), и для него используются отдельные инструменты — Flutter DevTools.

Эвристики и мнемоники



Эвристики - это быстрые, недорогие способы решения проблемы или принятия решения. В мире тестирования ПО мы можем использовать эвристики как ***выведенные опытным путём подсказки для принятия решений и решения проблем в ходе тестирования***. Они могут быть особенно полезными для генерации предположений, если мы не уверены, как начать тестирование, или исчерпали идеи, что делать дальше.

Мнемоника - это тип эвристики, набор правил и приемов, которые ***помогают эффективно запоминать необходимые сведения (информацию)***, обычно это слово-аббревиатура или фраза. Например, все помнят детскую мнемонику “каждый охотник желает знать где сидит фазан”, в которой по первым буквам каждого слова можно вспомнить порядок цветов радуги.

Множество известных тест-эвристик использует мнемоники и широко распространено заблуждение, что у эвристик должна быть мнемоника, или что это одно и то же. Это не так. Эвристики не требуют мнемоник, они просто создаются таким образом, чтобы их было легче запомнить.

Эвристики и мнемоники могут быть придуманы, модифицированы и скрещены как удобно автору для своих нужд. Далее показаны наиболее известные:

Эвристики и мнемоники

Мнемоники

I Inputs — Вводимое

S Store — Память

L Location — Местоположение

I Interruptions — Прерывания

C Communication — Взаимодействие

E Ergonomics — Эргономичность

D Data — Данные

U Usability — Удобство

P Platform — Платформа

F Functions — Функции

U User Scenarios — Сценарии

N Network — Сеть

L Location — Местоположение

O Orientation — Ориентация

N Network — Сеть

G Gestures — Жесты

F Functions — Функции

U User Scenarios — Сценарии

N Notifications — Уведомления

C Communication — Взаимодействие

U Updates — Обновления

P Platform — Платформа

Мнемоники

У БАГОВ ТОЖЕ ЕСТЬ ЧУВСТВА

НАЙДЯ БАГ,
СООБЩИТЕ О НЕМ

БАГИ НЕ ЛЮБЯТ
БЫТЬ ЗАБЫТЫМИ



НАЙДЯ БАГ,
ИЗУЧИТЕ ЕГО

БАГИ ЛЮБЯТ
БЫТЬ ПОНЯТЫМИ



У НЕЕ 3
ПЯТЫШКА

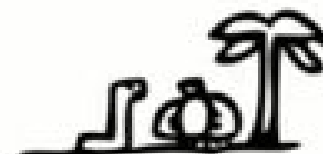
НАЙДЯ БАГ,
СДЕЛАЙТЕ СНИМОК

БАГИ ЛЮБЯТ ХРАНИТЬ
ПАМЯТЬ О ВСТРЕЧЕ



НАЙДЯ БАГ,
УЗНАЙТЕ ЕГО ДРУЗЕЙ

БАГИ ОБЩИТЕЛЬНЫ



НАЙДЯ БАГ,
СООБЩИТЕ О НЕМ БЫСТРО

ИНАЧЕ БАГИ ОБЖИВУТСЯ И
ПОСТРОЯТ СЕБЕ ДОМА



НАЙДЯ БАГ,
БУДЬТЕ ЧЕСТНЫ

БАГИ НЕ ЛЮБЯТ
СПЛЕТЕН



НАЙДЯ БАГ,
ЗАПОМНИТЕ КАК ВЫ
ВСТРЕТИЛИ ЕГО

БАГИ РОМАНТИЧНЫ



НАЙДЯ БАГ,
НЕ ИГНОРИРУЙТЕ ЕГО

БАГИ МОГУТ УКУСИТЬ,
ЕСЛИ ИХ НЕ ЦЕНИТЬ



Эвристики и мнемоники

Связь.

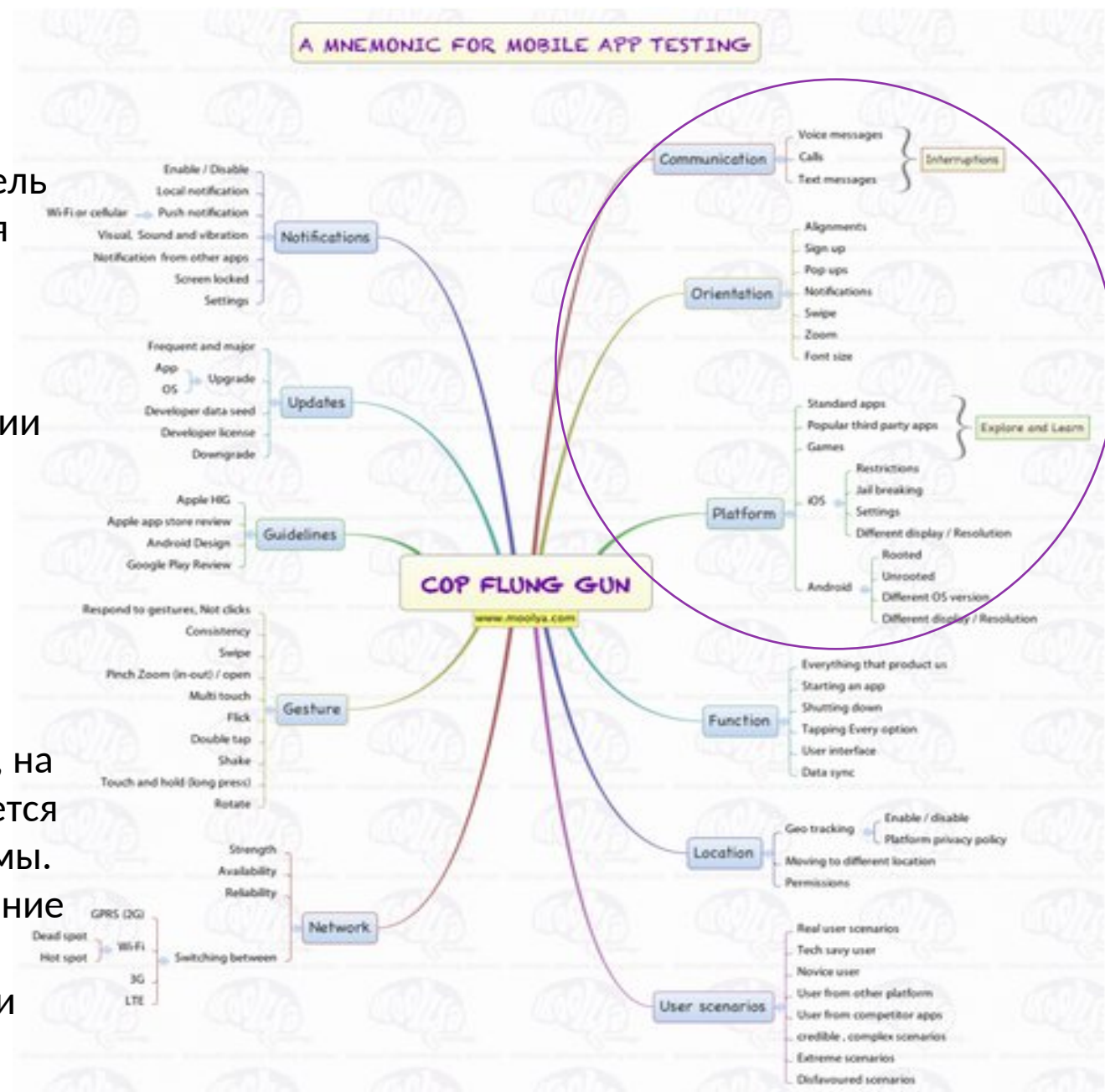
Это основная функция мобильного устройства. Эта модель предлагает проверить, как ваше приложение ведет себя при возникновении прерываний, таких как вызов, SMS, голосовые сообщения и т. д.

Ориентация:

проверьте поведение вашего приложения при изменении ориентации. Проверьте, как приложение работает при изменении ориентации при заполнении формы, при включенной клавиатуре, при прокрутке и т. д. Также обратите внимание на тесты эргономики и удобства использования.

Платформа.

Тестировщик должен хорошо разбираться в платформе, на которой он тестирует. Ключевым моментом здесь является использование стандартных приложений этой платформы. Тестировщики могут протестировать, например, поведение приложения на телефонах с рутинговым/нерутинговым/разлоченным доступом, с различными размерами экрана и версиями ОС и т. д.



Эвристики и мнемоники

Функция:

убедитесь, что приложение выполняет то, что оно должно делать с точки зрения требований. Попутно тестируйте сценарии вроде ввода данных в формы, изменения настроек, взаимодействия приложений.

Местоположение:

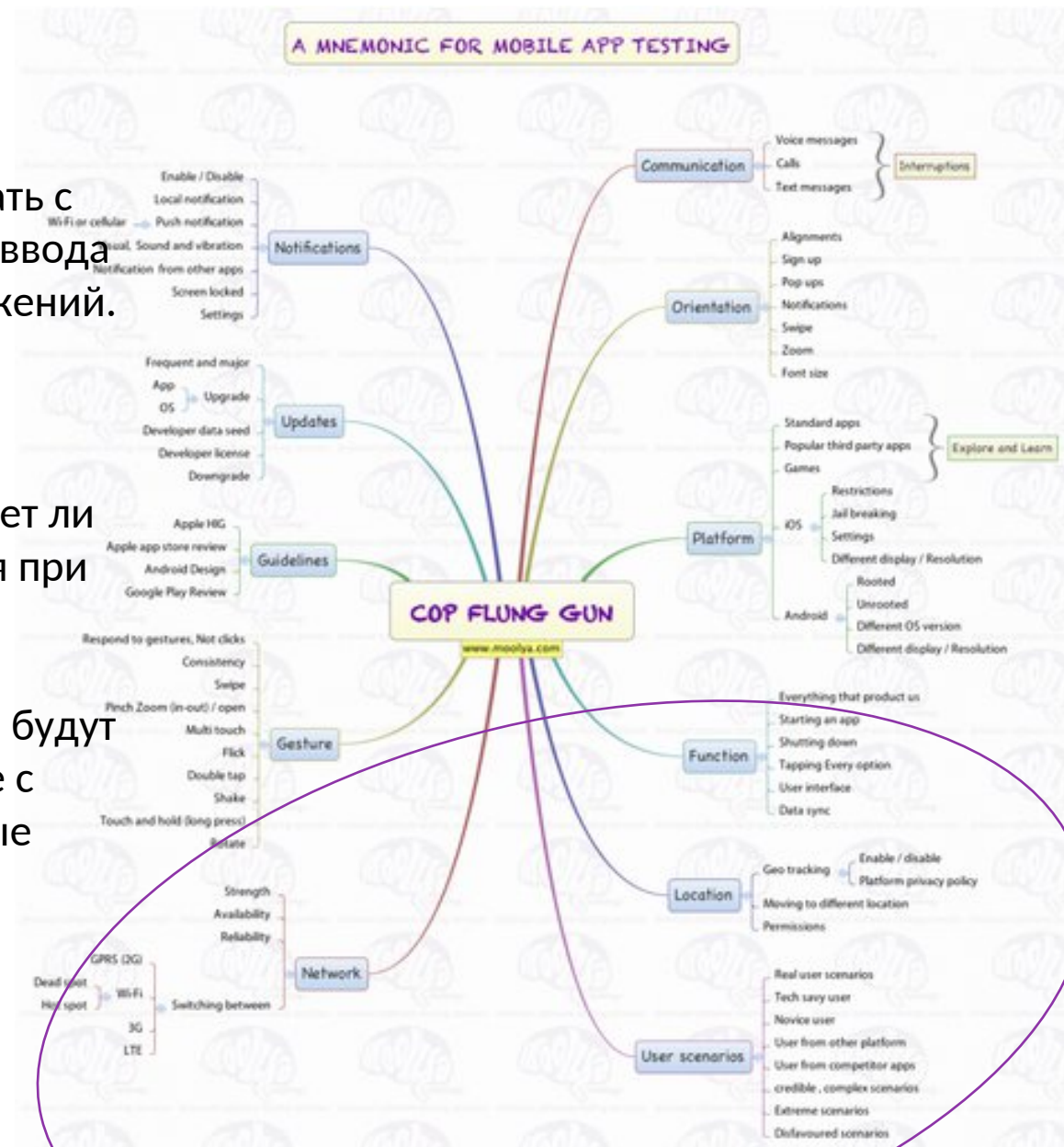
этот аспект предполагает никогда не тестировать мобильное приложение, сидя в одном месте. Проверьте отслеживание географического местоположения и его настройки, соответствует ли оно рекомендациям по отслеживанию, поведение приложения при выходе из соединения для передачи данных и т. д.

Пользовательские сценарии:

речь идет о продумывании реальных сценариев того, как люди будут использовать мобильное устройство и приложение. Подумайте с точки зрения конечных пользователей и проверьте, как опытные пользователи и новички будут использовать приложение.

Сеть.

Мобильные приложения сильно зависят от сети. Тестировщику необходимо проверить, как приложение ведет себя при переключении между Wi-Fi, GPRS, 3G и т. д., а также в других сценариях, связанных с сетью.



Эвристики и мнемоники

Жесты.

Если ваше приложение поддерживает пользовательские жесты, протестируйте их должным образом. Также проверьте наличие некоторых стандартных жестов, таких как пролистывание, мультитач, подтягивание обновления и т. д.

Рекомендации.

Поскольку для одобрения магазина приложений требуется соблюдение рекомендаций, убедитесь, что ваше приложение соответствует этим рекомендациям. Вам также необходимо отслеживать любые изменения в этих рекомендациях.

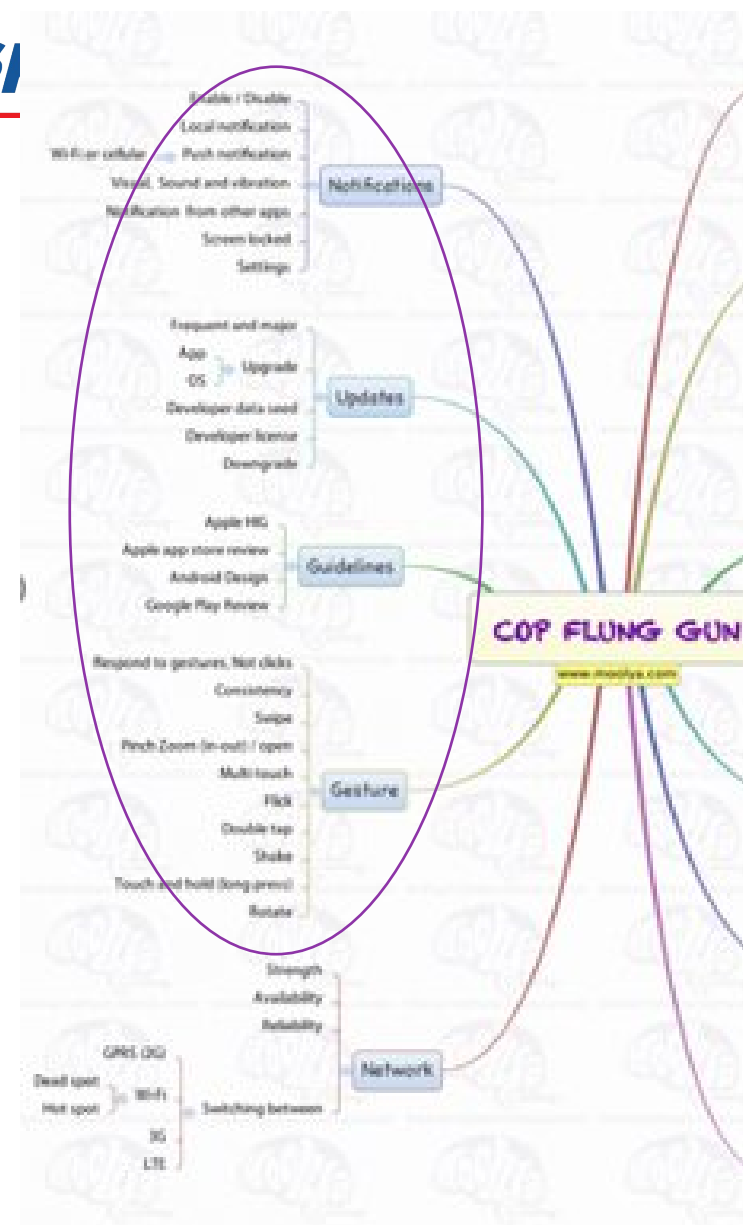
Обновления:

в этом аспекте предлагается проверить версию на одну более раннюю, чем последняя, и изучить ее. После этого вы можете обновиться до последней версии, чтобы проверить поведение приложения. Поскольку обновления мобильных приложений происходят часто, этот аспект имеет решающее значение для тестирования приложения.

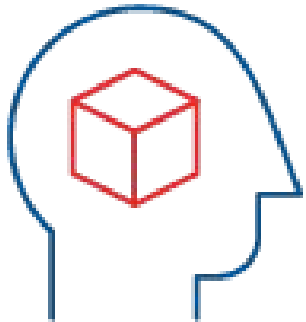
Уведомления:

проверьте, правильно ли работает уведомление приложения. Тестируйте уведомления в различных сценариях, например, на заблокированном экране, в различных сетевых условиях, в спящем режиме и т. д.

Итак, это «ингредиенты» модели COP FLUNG GUN.



Симуляторы и эмуляторы Тестовые фермы



Чтобы понять какой парк тестовых девайсов нам необходим, следует:

- ✓ выяснить особенности проекта (масштабность, аудиторию, охват устройств);
- ✓ проанализировать сроки и реалистично оценить нагрузку;
- ✓ посмотреть аналитику мобильных устройств пользователей и выявить приоритетные для тестирования модели.

Также нужно иметь в виду, что iOS поддерживает обновления ОС для довольно старых моделей. Например, последняя версия iOS 16 доступна более чем на 20 девайсах, начиная с iPhone 8. Поэтому тестированием на старых версиях iOS часто можно пренебречь. На Android же следует уделять внимание и прошлым версиям ОС, так как немалое количество пользователей с девайсами не самых свежих моделей, работающими, например, на Android 7-9.

Предположим, вы определились со списком желаемых девайсов. Осталось выяснить, где их найти?

Как тестировать мобильные приложения



Вариант 1: Симуляторы и эмуляторы

Симуляторы виртуально воспроизводят поведение системы и её интерфейса. Симуляции имитируют выполнение кода. В большинстве случаев, для запуска симулятора, используются XCode или Android Studio. **Эмуляторы** воссоздают все основные компоненты устройства, в том числе процессор, память и устройства ввода/вывода.

Главный недостаток как симуляторов, так и эмуляторов в том, что они **не могут полностью заменить физический девайс**. С их помощью мы можем проверить верстку или работоспособность приложения в целом, но мы не можем проверить корректную работу приложения при взаимодействии с остальными системами телефона, то есть как будет функционировать приложение при:

- ✓ медленном интернете;
- ✓ получении звонка или СМС;
- ✓ использовании камеры, микрофона или геолокации.

Работа приложения на симуляторе и эмуляторе может отличаться от работы на реальном девайсе ещё и по следующим причинам:

- ✓ при тестировании не учитывается аппаратная часть приложения;
- ✓ многое зависит от мощности компьютера, на котором выполняется тестирование.

Поэтому очень важно соблюдать баланс между физическими устройствами, симуляторами и эмуляторами. Как раз тут на помощь приходят тестовые фермы.

Вариант 2: Тестовые фермы

Тестовая ферма — это инструмент, который позволяет подключиться к реальному девайсу и выполнить полноценное тестирование. Экран устройства транслируется онлайн. При желании, тестовую ферму можно организовать и с личными устройствами.

Подборка тестовых ферм

При тестировании мы можем покрыть нехватку тестовых девайсов с помощью самых популярных тестовых ферм. Среди них есть бесплатные и платные варианты.

Как тестировать мобильные приложения

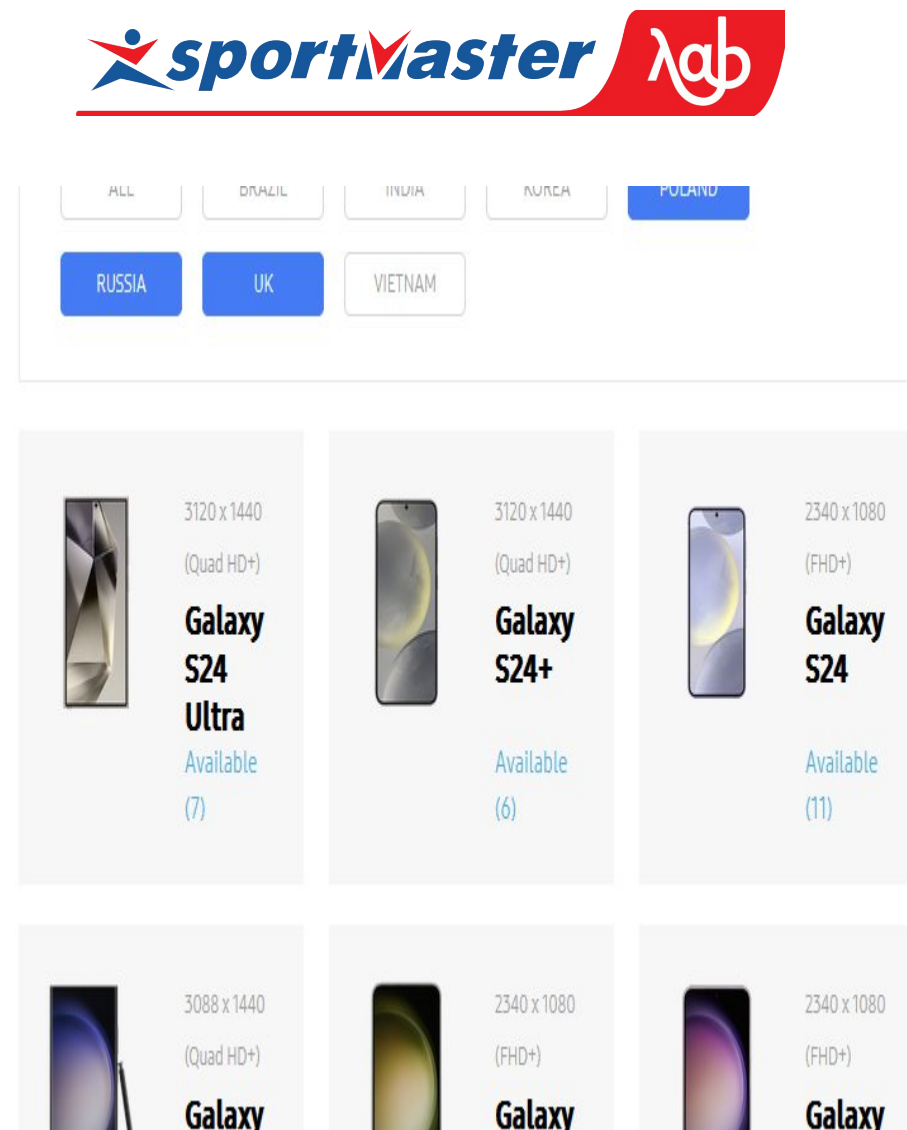
Вариант 2: Тестовые фермы

Samsung Test Lab

Бесплатная тестовая ферма с возможностью бронирования девайсов Samsung от 30 мин до 2 часов. Поддерживает режим удаленной отладки.

Как использовать:

Зарегистрировать устройство из списка, выбрать ОС и локацию, запустить удаленный доступ к экрану. Откроется либо веб-страница с доступом к девайсу, либо автоматически скачается файл с расширением .jnlp. При запуске файла на macOS, необходимо разрешить запуск из системных настроек, иначе компьютер ругается на безопасность (Системные настройки -> Защита и безопасность -> Разрешить). При первом запуске попросит установить пакеты Java.

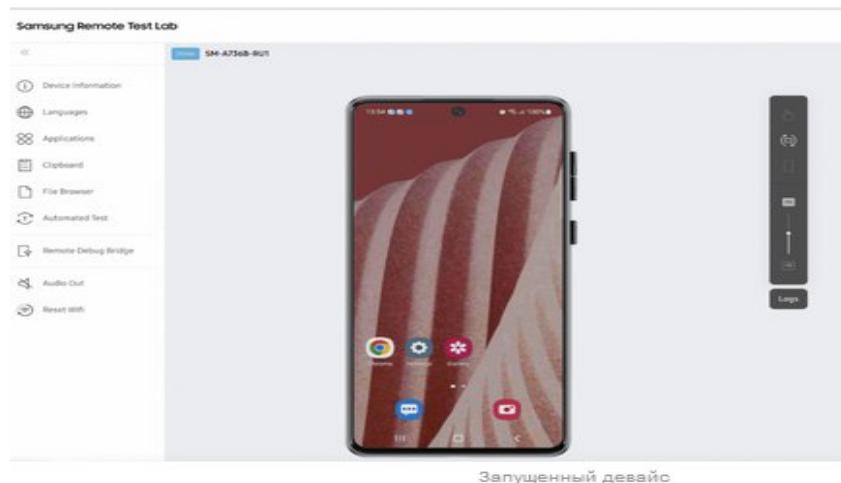


На сегодня доступ не закрыт

Как тестировать мобильные приложения

Вариант 2: Тестовые фермы

Samsung Test Lab



Huawei Cloud Debugging

Инструмент отладки приложений с возможностью удаленного доступа к девайсам. Перед каждым использованием устройство сбрасывается, что исключает проблемы с «забитостью» девайса.

Как использовать:

Перед использованием нужно зарегистрироваться. При регистрации просят загрузить документ, удостоверяющий личность, который действительно проверяют. Далее необходимо создать новый проект (Качество -> Облачная отладка) на боковой панели и зарезервировать интересующее устройство. В выбранное время устройство будет доступно на вкладке «Мои данные».



HUAWEI ID login

tings > HUAWEI ID,

Как тестировать мобильные приложения

Вариант 2: Тестовые фермы

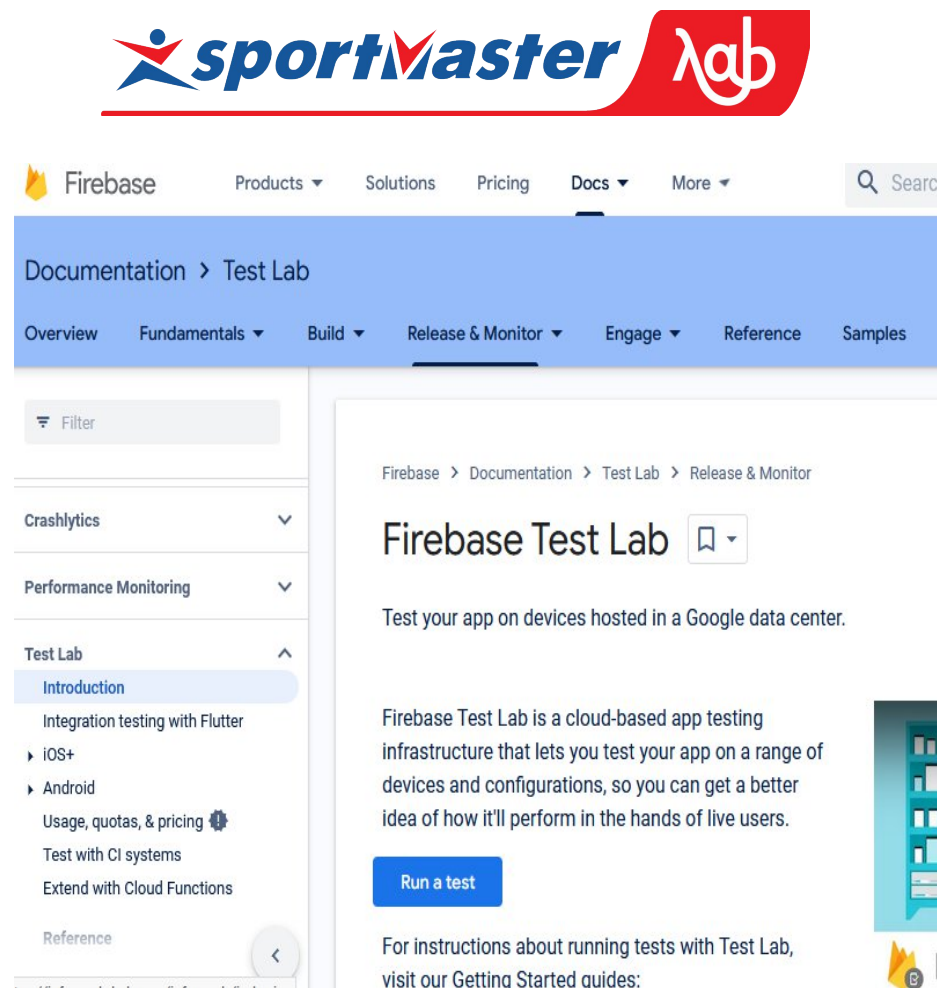
Платные тестовые фермы

Ниже приведены популярные платные тестовые фермы. Не могу поделиться опытом использования (пока что хватало бесплатных опций), но они могут пригодиться для ваших задач.

Firebase Test Lab — ферма от Google. Позволяет тестировать приложение на различных реальных устройствах и конфигурациях IOS и Android. Все устройства расположены в центре обработки данных Google. Test Lab интегрирована с консолью Firebase, Android Studio и с интерфейсом командной строки gcloud.

Бесплатно можно запустить 5 тестов в день на реальных устройствах и 10 тестов — на виртуальных. За большее придется платить 1\$ за час на эмуляторе и 5\$ за час на реальном устройстве.

<https://habr.com/ru/companies/sbermarket/articles/690906/>



На сегодня доступ не закрыт

Как тестировать мобильные приложения

Вариант 2: Тестовые фермы



Преимущества:

- Нет необходимости покупать большое количество реальных девайсов.
- Можно найти устройство практически для любой ситуации.
- Нет затрат на поддержку и ремонт тестовых устройств.

Недостатки:

- Сетевая задержка, не позволяющая корректно отображать анимации.
- Работать как с полноценной заменой физических устройств тоже проблематично из-за отсутствия тачскрина.
- Тестировщики обычно работают сразу с несколькими устройствами, перепрыгивая с одного на другое и возвращаясь обратно. С фермами такое сложно.
- Безопасность данных под вопросом. В случае облачных ферм вы тестируете приложение на ресурсах другой компании, нет гарантии, что ваши данные не попадут куда-то еще.

Как тестировать мобильные приложения



Вариант 2: Тестовые фермы

Симуляторы, эмуляторы и тестовые фермы — что выбрать?

Эмуляторы, симуляторы и тестовые фермы дополняют друг друга и могут быть полезными для различных целей. В СберМаркете, например, нашли золотую середину и используют все типы инструментов для разных типов тестирования.

Для **UI тестирования** используют реальные девайсы и симуляторы, так как при проведении UI-тестов важно задействовать большой набор девайсов с различными версиями ОС, разрешением и диагональю экрана, а симуляторы для этого являются самым доступным инструментом.

Для выполнения **исследовательского тестирования и проверки корнер-кейсов** используют реальные девайсы и девайсы тестовых ферм. Исследовательское тестирование в большинстве случаев проводится для воспроизведения того или иного бага, поэтому необходимо придерживаться характеристик девайса, на котором баг был обнаружен. А так как данного девайса может не быть в распоряжении тестировщика, на помощь приходят девайсы тестовых ферм.

Для выполнения основных **пользовательских сценариев и регрессионного тестирования** используем реальные девайсы и симуляторы, так как основные сценарии, как правило, отрабатывают корректно на большом пуле устройств и не зависят от специфических характеристик девайса.

Если разумно подойти к анализу и последующему подбору необходимых тестовых устройств и грамотно использовать при тестировании симуляторы, эмуляторы, и тестовые фермы, то качество и эффективность тестирования продукта на удаленке, не будет отличаться от тестирования, когда команда сидит в одном кабинете.

<https://habr.com/ru/companies/sbermarket/articles/690906/>

Задание 1 (Mobile)



Скачать и протестировать мобильные приложения из подборки ниже.
Протестировать для 2-х версий OS Android:

1) С использованием какой-то из тестовых ферм.

- Android 13: 15% → 22,4%;
- Android 12: 16,3% → 15,8%;
- Android 11: 23,1% → 21,6%;
- Android 10: 17,8% → 16,1%;
- Android 9: 11,9% → 10,5%;
- Android 8.1: 6,1% → 5,4%;
- Android 8: 2,2% → 1,9%;
- Android 7.1: 1,5% → 1,3%;
- Android 7: 1,5% → 1,3%;
- Android 6: 2,3% → 1,9%;
- Android 5.1: 1,5% → 1,2%;
- Android 5: 0,3% → 0,2%;
- Android 4.4: 0,5% → 0,4%.

Продуктовые магазины →



Мой SPAR — продукты и... 4,7 ★
Чижик - продукты 3,2 ★
Магнит: акции и до... 4,6 ★

Продуктовые магазины →



Globus — гипермарк... 3,6 ★
Перекрёсток Впрок гипер... 4,4 ★
Жизнь март. Доставка п... 4,7 ★

Продуктовые магазины →



Хороший Выбор Онл... 3,5 ★
Open Food Facts 2,5 ★
Яндекс Лавка: зака... 4,8 ★

Рестораны и доставка еды →



Деливери: еда и прод... 3,3 ★
Жизнь март. Доставка п... 4,7 ★
DOSTAEBCK ИЙ — Дост... 2,1 ★

Задание 1 (Mobile)

Оформить отчет согласно чек-листу (образец чек-листа будет выдан):

TestCases_for_MobileApps.xlsx * X									
A	B	C	D	E					
1									
2	Application Name	<mention application name here>	Version	<version of the application>					
3									
4	Testing Type	<manual/automation>	Testing Start Date	<testing start date>					
5									
6	OS_ver	<chrome/IE/safari/others>	Testing End Date	<testing end date>					
7									
8	Browser version	<version of the browser on which testing is done>	Cycle No.	<1/2/3>					
9									
10	FrontEnd Server Name	<frontend server name on which testing is done>	CR Status	<open/close/deferred/hold>					
11									
12	BackEnd Server Name	<backend server name on which testing is done>	Prepared By						
13									
14	DB Name								
15									
16									

TestCases_for_MobileApps.xlsx * X								
A	B	C	D	E	F	G	H	
1								
2	Case No.	Description	Menu Name	Test Cases	Test Steps	Test Data	Expected Output	Actual Output
3	1	Header		Mobile app Test Cases				
4	2	Observation	Mobile app	Verify by installing the application from Google Play Store / App Store.			The application should be Installed successfully.	
5	3	Observation	Mobile app	Verify by installing the application in mobile using .apk/.ipa file in android/iphone devices respectively.			The application should be Installed successfully.	
6	4	Observation	Mobile app	Check for the display of App icon on the Home Screen when app is installed successfully.			After successful installation, App icon should be displayed on the Home Screen of the mobile device.	
7	5	Observation	Mobile app	Verify by uninstalling the application from Mobile device.			The application should be Uninstalled successfully and all the data related to the app should also be removed.	

TestCases_for_MobileApps.xlsx

Задание 2 (Mobile)



Протестировать какой-либо интернет ресурс в браузере мобильного устройства

С использованием DevTools.

Задание 2

Оформить отчет согласно образцу чек-листа (образец чек-листа будет выдан):

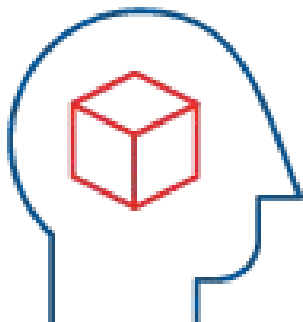
Mobile_testing.xlsx					
	A	B	C	D	E
1	Яндекс Метро версия 3.6				
2	№	Описание	Статус	ID Баг-репорта	Требование
3	1. Список маршрутов				
4	Кнопка Детали маршрута				
5	1	Кнопка "Детали маршрута" располагается в соответствии с макетом.	PASSED		1.1
6	2	Кнопка "Детали маршрута" соответствует макету по дизайну.	PASSED		1.1
7	3	Кнопка "Детали маршрута" не имеет орфографических ошибок.	PASSED		1.1
8	4	Кнопка "Детали маршрута" не отображается при нажатии на крест в поле ввода маршрута "Откуда".	PASSED		1.1
9	5	Кнопка "Детали маршрута" не отображается при нажатии на крест в поле ввода маршрута "Куда".	PASSED		1.1
10	6	Кнопка "Детали маршрута" не отображается при нажатии на крест в правом верхнем углу в панели "Детали маршрута".	PASSED		1.1
11	7	Кнопка "Детали маршрута" отображается при нажатии на кнопку "Поменять местами начальную и конечную станцию".	PASSED		1.1
12	Обновление временного интервала маршрута, если текущее время превышает время окончания маршрута.				
13	8	Временной интервал маршрута не обновляется, если текущее время меньше времени окончания маршрута на 1 минуту.	PASSED		1.2
14	9	Временной интервал маршрута не обновляется, если текущее время равно времени окончания маршрута.	PASSED		1.2
15	10	Временной интервал маршрута обновляется, если текущее время больше времени окончания маршрута на 1 минуту.	FAILED	BUG-91702	1.2
16	11	Временной интервал маршрута обновляется, если текущее время больше времени окончания маршрута на 2 минуты.	FAILED	BUG-91702	1.2
17	2. Выбор станции.				
18	Станция выбирается тапом по схеме.				
19	12	Выбор станции тапом по схеме.	PASSED		2.1
20	Станция выбирается тапом по иконке i.				
21	13	Выбор станции тапом по иконке i в карточке поля "Откуда".	PASSED		2.1
22	14	Выбор станции тапом по иконке i в карточке поля "Куда".	PASSED		2.1
23	15	Выбор станции тапом по иконке i в карточке детали маршрута.	PASSED		2.1

Поскольку приложения разные, то добавляется задача скорректировать чек-лист, согласно приложению.

Mobile_testing.xlsx

Android Debug Tool (ADB) и Logcat

Инструменты для снятия логов: Android



Инструменты для снятия логов: Android



Существует 3 способа:

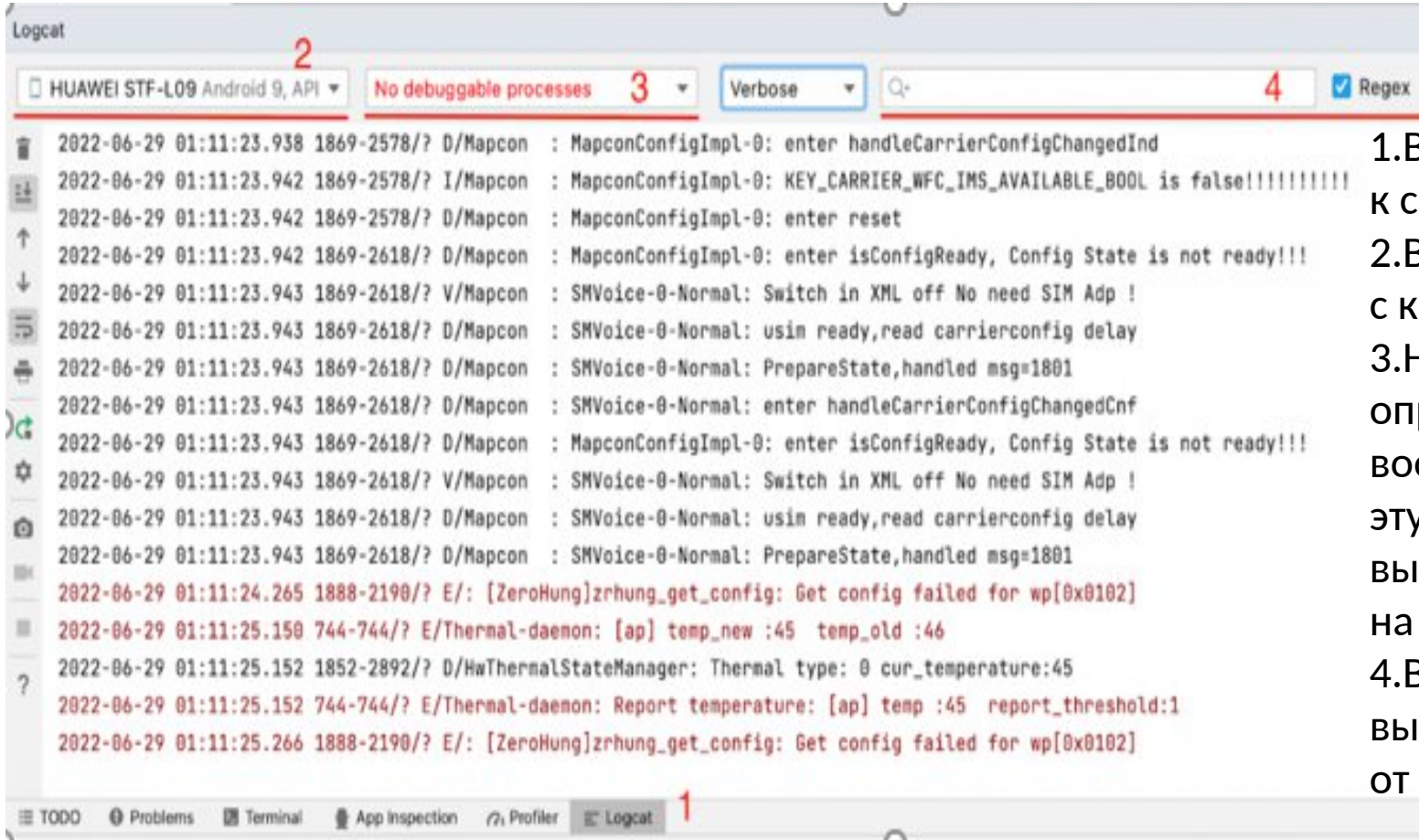
- 1 Первый — Logcat** в составе Android Studio, самый известный и широко используемый. Для снятия логов нам необходимо перевести устройство в режим разработчика/отладки.
Примечание: алгоритм может отличаться в зависимости от производителя устройства, потому что у многих из них свои надстройки на ОС Android.
- 2 Второй способ — выгрузка логов с самого устройства.** Кроме режима разработчика нам нужно подключить устройство к ПК через USB и установить ADB — [Android Debug Bridge](#).
- 3 Третий инструмент — SDK Platform Tools.** Процесс его установки практически аналогичен предыдущим двум.

Инструменты для снятия логов: Android Studio

Дальше подключаем устройство по USB к ПК и устанавливаем Android Studio.

Следующие шаги на скрине:

1

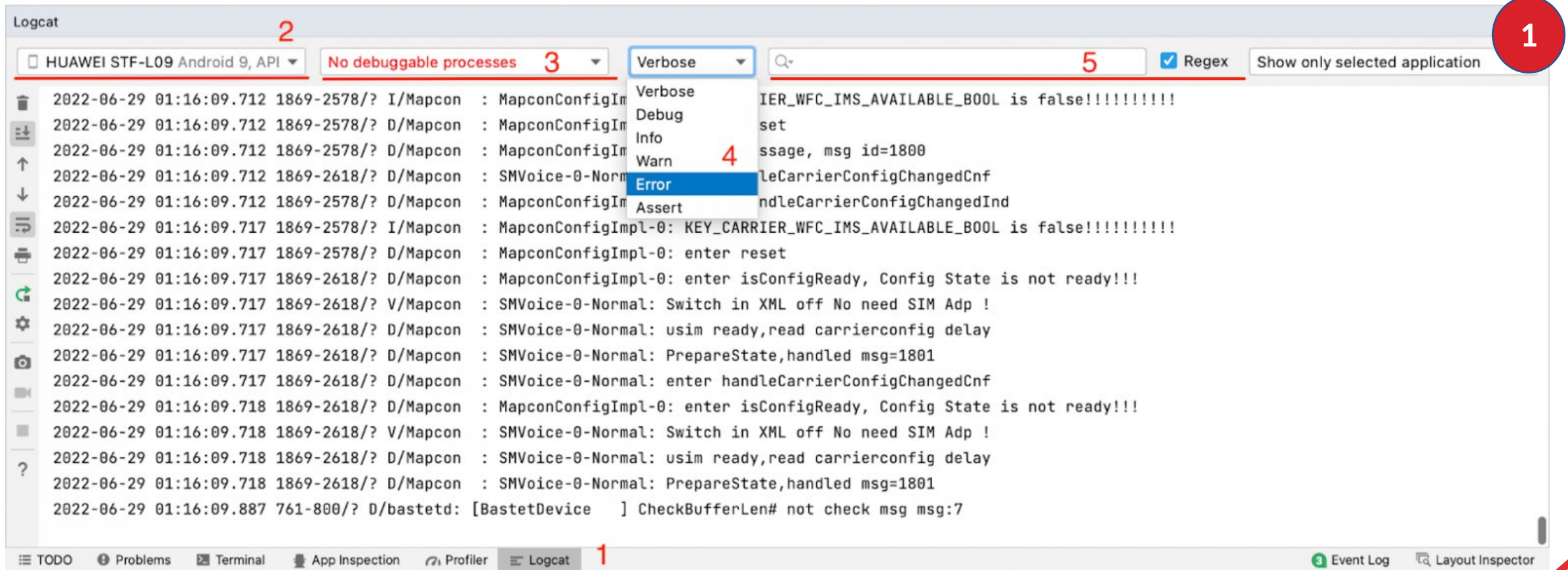


1. Выбираем вкладку Logcat (переходим к сообщениям в реальном времени).
2. В окошке выбираем телефон (устройство), с которого снимаем логи.
3. На этой вкладке выбираем логи определённого приложения. Если нужно снять вообще все логи со всех приложений и системы, эту вкладку стоит не трогать. Рядом с ней можно выбрать уровень логирования (вкладка Verbose на скрине).
4. В поле поиска, где мы можем фильтровать выдачу, разрешено писать что угодно — от названия пакета до частей вроде fatal.

Инструменты для снятия логов: Android Studio



На скрине видно логи с подключенного устройства.



Инструменты для снятия логов: Android Studio



Android Studio

Pixel XL API 30

Pixel 4 API 30

Virtual Device Configuration

Select Hardware

Choose a device definition

Category	Name	Play Store	Size	Resolution	Density
TV	Pixel XL		5.5"	1440...	560dpi
Phone	Pixel 4 XL		6.3"	1440...	560dpi
	Pixel 4		5.7"	1080...	440dpi
Wear OS	Pixel 3a XL		6.0"	1080...	400dpi
	Pixel 3a		5.6"	1080...	440dpi
Tablet	Pixel 3 XL		6.3"	1440...	560dpi
	Pixel 3		5.46"	1080...	440dpi
Automotive	Pixel 2 XL		5.99"	1440...	560dpi

New Hardware Profile

Import Hardware Profiles

Pixel XL

1440px

5.5"

2560px

Size: large

Ratio: long

Density: 560dpi

Clone Device...

Cancel

Previous

Next

Finish

Watch later

Share

1

Android Debug Bridge (adb) — это инструмент программирования, используемый для отладки устройств на базе Android.

Daemon на устройстве Android подключается к серверу на хост-компьютере через USB который, в свою очередь, подключается к клиенту, используемому конечным пользователем, через TCP.

Возможности adb включают:

- ✓ копирование файлов с главного компьютера,
- ✓ установку приложений,
- ✓ просмотр вывода logcat,
- ✓ получение оболочки Unix,
- ✓ перезагрузку в режиме Qualcomm EDL.

Программное обеспечение adb совместимо с Windows, Linux и macOS.



Android Debug Tool (ADB)

Качаем отсюда архив <https://adbinstaller.com/> , распаковываем, получаем:

2

Сегодня

platform-tools_r35.0.0-windows.zip

https://adbinstaller.com

Расписание ФКТиПМ ⚙ Часто посещаемые Android JAVA База информационн... BigData

ADB Installer

Home ADB Commands AAPT Command

Android Debug Bridge (adb) is a versatile command-line tool that lets you communicate with a variety of device actions, such as installing and debugging apps, and it provides access to a variety of commands on a device. It is a client-server program that includes three components:

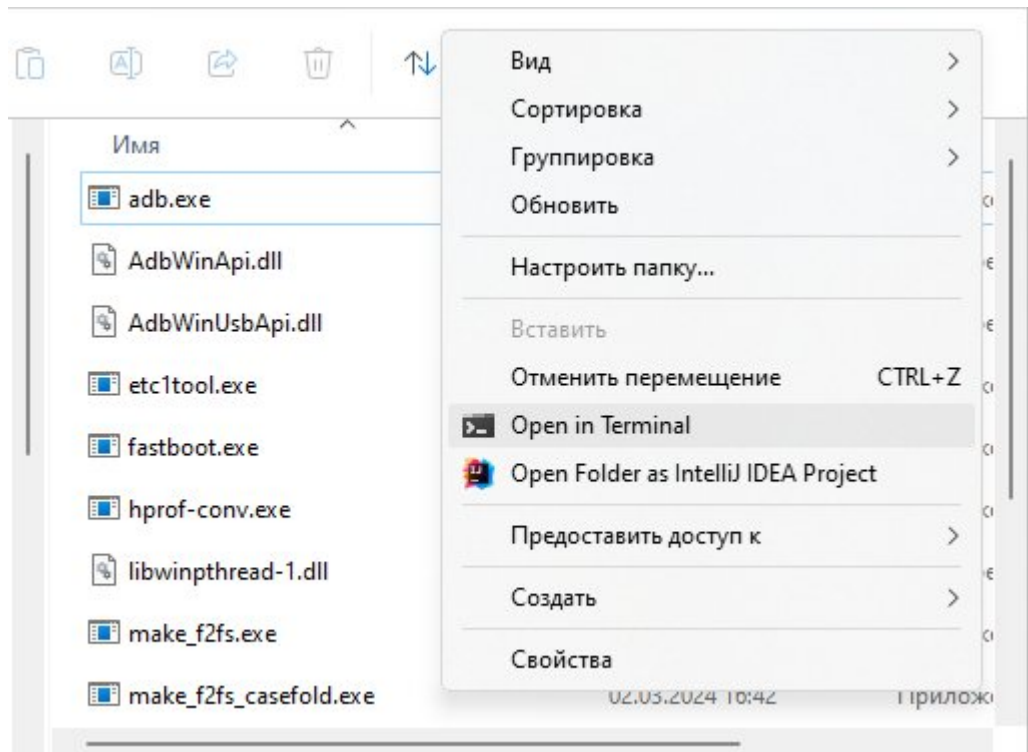
ADB and fastboot platform-tools

OS	Details	Downloads
Windows	What's Included?	Download (4.55 MB)
Linux	What's Included?	Download (5.22 MB)
macOS	What's Included?	Download (5.19 MB)

Имя	Дата изменения	Тип	Размер
adb.exe	02.03.2024 16:42	Приложение	5 712 КБ
AdbWinApi.dll	02.03.2024 16:42	Расширение при...	106 КБ
AdbWinUsbApi.dll	02.03.2024 16:42	Расширение при...	72 КБ
etc1tool.exe	02.03.2024 16:42	Приложение	429 КБ
fastboot.exe	02.03.2024 16:42	Приложение	1 762 КБ
hprof-conv.exe	02.03.2024 16:42	Приложение	54 КБ
libwinpthread-1.dll	02.03.2024 16:42	Расширение при...	237 КБ
make_f2fs.exe	02.03.2024 16:42	Приложение	467 КБ
make_f2fs_casefold.exe	02.03.2024 16:42	Приложение	467 КБ
mke2fs.conf	02.03.2024 16:42	Файл "CONF"	2 КБ
mke2fs.exe	02.03.2024 16:42	Приложение	737 КБ
NOTICE.txt	02.03.2024 16:42	Text Document	1 085 КБ
source.properties	02.03.2024 16:42	Файл "PROPERTIES"	1 КБ
sqlite3.exe	02.03.2024 16:42	Приложение	2 772 КБ

Android Debug Tool (ADB)

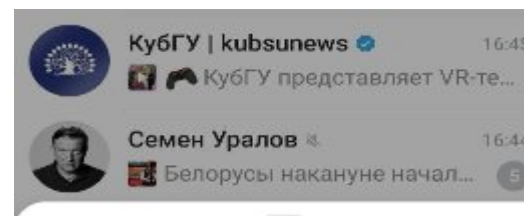
Находясь в директории ADB, запускаем терминал и пользуемся:



```
PS C:\adb> .\adb devices
List of devices attached
2c15910b          device

PS C:\adb> .\adb logcat -b events
```

! Но прежде подключаете ваш смартфон по USB и включаете режим разработчика:



Использовать USB для

- ☐ Только зарядка
- ☒ Перенос файлов / Android Auto
- ☐ Перенос фото



Если устройство не распознается, то попробуйте переподключить USB еще раз.

Android Debug Tool (ADB)

```
PS C:\adb> .\adb devices
List of devices attached
2c15910b          device

PS C:\adb> .\adb logcat -b events|
```

Если вы нигде не ошиблись, то увидите в командной строке сообщение «List of devices attached» с указанием номера вашего смартфона, подключенного к ПК через Android Debug Bridge. В дальнейшем для взаимодействия с отладочным мостом вам понадобятся **команды ADB**, которые нужно вводить аналогичным образом через командную строку или PowerShell.

```
03-02 17:18:18.215 2071 2668 I power_screen_broadcast_done: [0,378,1]
03-02 17:18:18.262 2071 3110 I service_manager_stats: [100,59,62552]
03-02 17:18:18.290 2071 3052 I commit_sys_config_file: [settings-1-0,6]
03-02 17:18:22.438 2071 4589 I wm_set_keyguard_shown: [1,0,0,setKeyguardShown]
03-02 17:18:22.499 3584 3584 I sysui_multi_action: [757,400,758,4,806,org.telegram.messenger]
03-02 17:18:22.502 3584 3584 I sysui_multi_action: [757,196,758,1,759,1]
03-02 17:18:22.503 3584 3584 I sysui_status_bar_state: [1,1,0,0,1,0]
03-02 17:18:22.505 3584 3584 I sysui_multi_action: [757,400,758,4,806,org.telegram.messenger]
03-02 17:18:22.519 3584 3584 I sysui_multi_action: [757,111,758,1]
03-02 17:18:22.519 3584 3584 I sysui_multi_action: [757,126,758,1,759,126,927,0,928,1,1593,0]
03-02 17:18:22.519 3584 3584 I sysui_multi_action: [757,115,758,1,759,115,927,1,928,1,1593,0]
03-02 17:18:22.519 3584 3584 I sysui_multi_action: [757,119,758,1,759,119,927,2,928,1,1593,0]
03-02 17:18:22.519 3584 3584 I sysui_multi_action: [757,113,758,1,759,113,927,3,928,0,1593,0]
03-02 17:18:22.519 3584 3584 I sysui_multi_action: [757,119,758,1,759,119,927,4,928,0,1593,0]
03-02 17:18:22.519 3584 3584 I sysui_multi_action: [757,122,758,1,759,122,927,5,928,0,1593,0]
03-02 17:18:22.569 2071 2445 I user_activity_timeout_override: 10000
```

Android Debug Tool (ADB)

Вводим команду — **adb -s название устройства logcat**, — которая запускает утилиту Logcat для конкретного устройства. В терминале в реальном времени будут поступать логи.

```
Windows PowerShell
Windows Terminal can be set as the default terminal application in your settings. Open Settings

03-02 18:37:56.166 2071 3199 D HoraeHelper: skinThermal:27.87
03-02 18:37:56.167 2071 3199 I chatty : uid=1000(system) HwBinder:2071_2 identical 3 lines
03-02 18:37:56.168 2071 3199 D HoraeHelper: skinThermal:27.87
03-02 18:37:56.169 2071 2910 D HoraeHelper: skinThermal:27.87
03-02 18:37:56.416 2071 3203 D JobScheduler.Temperature: get temperature=27.87
03-02 18:37:56.590 2071 4593 V UserManagerService: dumpPackageWhitelistProblems(): using mode ENFORCE|I
ST
03-02 18:37:56.721 2071 3199 D HoraeHelper: skinThermal:27.87
03-02 18:37:56.723 2071 3199 D BatteryLed: getLightInColorState, oppoLightMgr empty!
03-02 18:37:56.722 2071 3199 I chatty : uid=1000(system) HwBinder:2071_2 identical 3 lines
03-02 18:37:56.722 2071 3199 D HoraeHelper: skinThermal:27.87
03-02 18:37:56.725 2071 2910 D HoraeHelper: skinThermal:27.87
03-02 18:37:56.738 4120 4120 D DeviceStatisticsService: chargerType=2 batteryLevel=98 totalBatteryCapac
03-02 18:37:56.738 4120 4120 D DeviceInfoHidlClient: isRadioOn()=true
03-02 18:37:56.738 4120 4120 I DeviceInfoHidlClient: isPowerInfoNeverSent=false batteryLevel=98 battery
argingMode=2 totalCapacity=4347900 powerSaveMode=false
03-02 18:38:00.183 1081 1099 D vendor.oplus.hardware.charger@1.0-service: charging pthread 94209
03-02 18:38:00.945 19496 19541 D HoraeHelper: getAllShellTemps tSize : 4
03-02 18:38:00.946 19496 19541 D HoraeHelper: getAllShellTemps tSize : 4
03-02 18:38:01.775 3527 3660 D QtiRadioIndication: onSignalStrengthChange: slotId = 1
03-02 18:38:01.775 3527 3660 D QtiRadioIndication: onSignalStrengthChange: {rsrp = -32768, .snr = -32768}
03-02 18:38:01.775 3527 4514 D MainServiceImplHandler: : EVENT_ON_5G_SIGNAL_STRENGTH_CHANGE_IND
03-02 18:38:01.776 3584 26174 D FiveGServiceClient: onSignalStrength: slotId=1 token=Token: -1 status=Status: 1 signalS
trength=SignalStrength: Rsrp: -32768 Snr: -32768
03-02 18:38:01.785 3527 3527 E OplusNecManagerHelper: com.oplus.nec.OplusNecManager.notifyOrigSignalStrengthChanged [c
lass android.telephony.SignalStrength, int]
```

1. В первом столбце — дата и время поступления записи.
2. Во втором — обозначения уровней логирования. Например, D — это Debug.
3. В третьем показываются названия инструмента, утилиты, пакета, от которых поступает сообщение, а также расшифровка того, что вообще происходит.

Третий инструмент — SDK Platform Tools. Процесс его установки практически аналогичен предыдущим двум:

- ✓ переводим телефон в режим разработчика,
- ✓ подключаем к ПК по USB,
- ✓ скачиваем на ПК папку SDK PT (под свою ОС),
- ✓ открываем папку SDK PT в терминале.

Теперь пишем команду **`./adb logcat -v threadtime > ./android-debug.log`**.

Logcat в консоли

Если по каким-либо причинам вы не хотите открывать или устанавливать Android Studio, то logcat можно также запускать из командной строки (терминала). Для этого есть команда `adb logcat`. Убедитесь, что путь до `adb` добавлен в переменные окружения, иначе система скажет что она не знает такой команды.

Запуск logcat из терминала также удобен для тестировщиков. Если есть баг или креш, который нужно зафиксировать, то можно

- ✓ Запустить команды `adb logcat > my_logs.txt` в консоли
- ✓ Воспроизвести ошибку
- ✓ Остановить выполнения логирования (например по `ctrl + c`)
- ✓ Скинуть логи из файлы `my_logs.txt` разработчикам.

Уровни логирования в Logcat

Для удобства выделены несколько возможных уровней логирования. В зависимости от важности и критичности вывода можно использовать те или иные приоритеты:

- ✓ `Log.e(String, String)` (ошибки)
- ✓ `Log.w(String, String)` (предупреждения)
- ✓ `Log.i(String, String)` (информация)
- ✓ `Log.d(String, String)` (дебаг)
- ✓ `Log.v(String, String)` (подробная информация)

Logcat

```
\adb logcat > logcat.txt
```

Все события будут записываться непрерывно по мере работы устройства. Остановить запись можно стандартной комбинацией Ctrl + C. Однако в лог попадает вся информация, что сильно затрудняет поиск нужной. Поэтому для работы обычно используют набор ключей и фильтров, подходящих к конкретной ситуации. Существует семь приоритетов сообщений по мере возрастания: V — Verbose, D — Debug, I — Info, W — Warning, E — Error, F — Fatal, S — Silent. Например, для вывода всех сообщений с приоритетом E и выше следует ввести:

```
\adb logcat *:E
```

После этого можно запускать проблемное приложение и смотреть, что именно вызывает ошибку. Также поддерживается вывод информации из альтернативных буферов. Этим способом можно посмотреть, что приложения делают в фоне и, например, какие события происходят после включения экрана:

```
\adb logcat -b events
```


Logcat

Фильтровать сообщения по приоритету можно следующим образом.

```
\adb logcat '*:W'
```

Список параметров команды logcat:

- d вывести лог на экран и выйти;
 - b загрузить альтернативный буфер лога для просмотра, такой как event или radio;
 - c очистить лог и выйти;
 - f записать сообщения лога в файл. По умолчанию в stdout;
 - g вывести размер указанного буфера лога и выйти;
 - n установить максимальное количество ротаций лога;
 - s устанавливает silent фильтром по умолчанию;
 - v устанавливает формат вывода сообщений лога. По умолчанию brief.
- С полным списком команд logcat можно ознакомиться на сайте developer.android.com.

Продвинутый уровень

С помощью Logcat можно даже доставать информацию из баз данных различных приложений. Посмотрим, как проделать это прямо из консоли, не качая базы на комп и не устанавливая на устройство просмотрщика баз. Для этого используется команда **sqlite3**. Выведем на экран историю браузера Chrome.

```
$ cd /data/data/com.android.chrome
$ su
# sqlite3 app_chrome/Default/History
> .schema urls
> select * from urls where url like "%android%";
```

Чтобы база читалась, необходимо выгрузить браузер из работающих приложений. Прервать выполнение скрипта sqlite можно, нажав Ctrl + Z, а выйти — командой .quit. Если в ответ на команду получена ошибка /system/bin/sh: sqlite3: not found, значит, на смартфоне нет sqlite3 и ее придется скачать, закинуть в /system/bin и дать файлу все права.

Продвинутый уровень

Также с помощью sqlite3 можно выдернуть все контакты с телефона. Для этого в консоли на компе должен использоваться шрифт Lucida Console и перед началом выполнения команд необходимо перевести кодировку на UTF-8. Иначе вместо русских букв будут отображаться непонятные символы. Сами команды выглядят так:

```
chcp 65001
adb shell
$ su # cd /data/data/com.android.providers.contacts/databases
# sqlite3 contacts2.db
> select t1.raw_contact_id,t1.normalized_number,t2.display_name from phone_lookup as t1,
raw_contacts as t2 where t1.raw_contact_id=t2._id Order by display_name;
```

Если все сделано правильно, то в консоли ты увидишь таблицу с порядковым номером записи, номером телефона и контактами, отсортированными по имени. Для контактов с более одного номера будет несколько записей подряд.

Очевидно, что с помощью ADB можно сделать много интересного. И чем больше пользуешься консолью, тем быстрее можно выполнить некоторые действия без установки дополнительного софта на устройство.

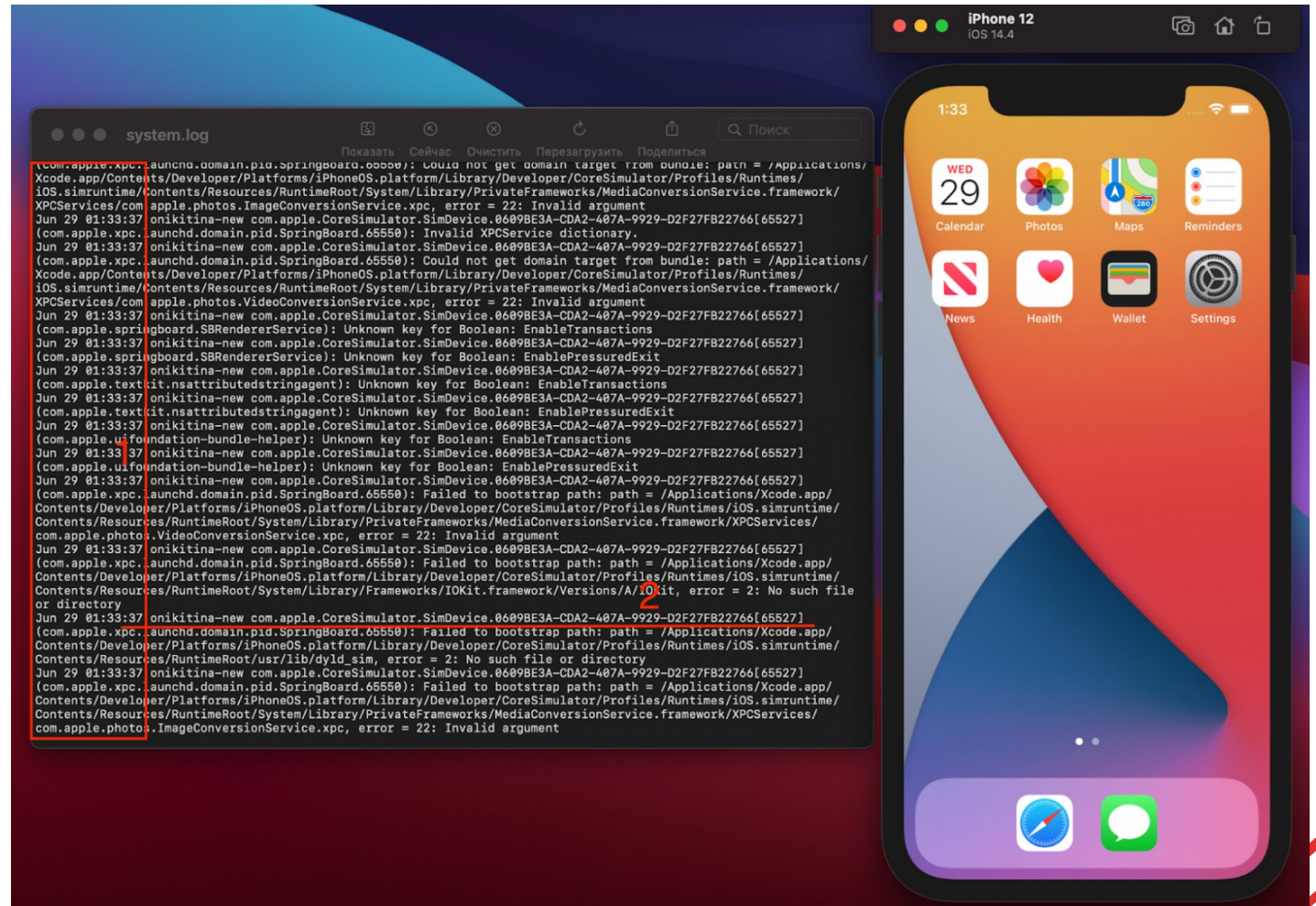
Инструменты снятия логов: iOS

В первую очередь нас интересует **xCode** — интегрированная среда разработки (IDE), в которую встроен нужный нам инструмент **Simulator**.

Как использовать инструмент:

1. Устанавливаем xCode.
2. В системной строке нажимаем xCode → Open Developer Tools → Simulator.
3. Устанавливаем приложение.
4. В самом симуляторе выбираем Debug → Open System Log.

Мы будем видеть логи в реальном времени:

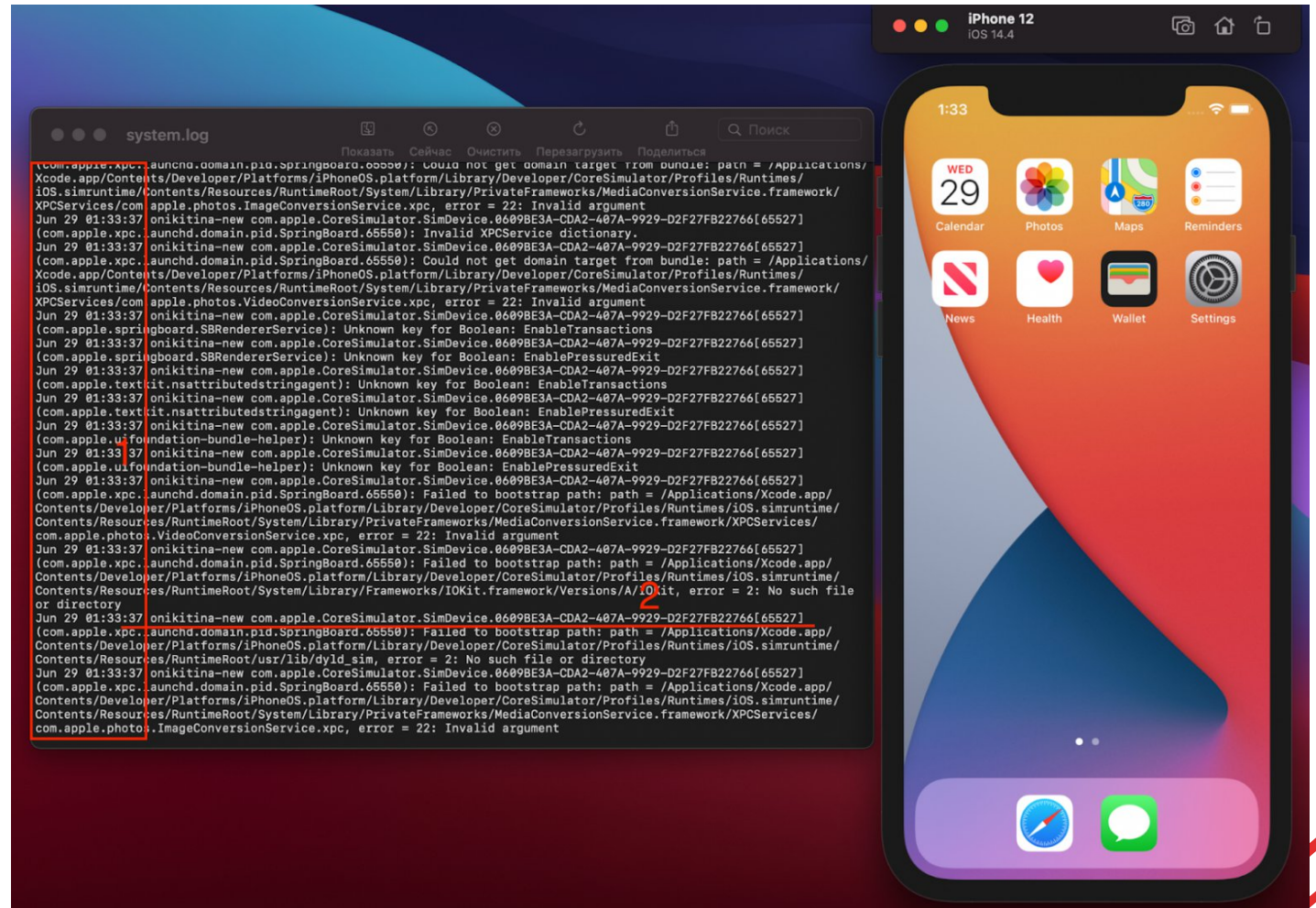


Инструменты снятия логов: iOS

Подобное оформление логов мы уже где-то видели, но построение информации в выдаче немного отличается. Есть дата и время (1) и данные (2) о том, с какого устройства снята информация: имя компьютера, элемент системы, с которого пришло сообщение, и его расшифровка.

Но первый способ работает только с симуляторами. Если необходимо снимать логи с реального устройства, в этом может помочь раздел **Devices and Simulators**.

Записи можно отфильтровать по конкретному процессу (вашему приложению).



Подробнее смотрите здесь:

<https://habr.com/ru/companies/redmadrobot/articles/687184/>

Задание 3 (Mobile)



Установить ADB, подключить телефон, посмотреть логи с помощью Logcat, результаты отобразить в отчете.

1. Консольный Android: 50 команд, о которых должен знать каждый.
<https://xakep.ru/2016/05/12/android-adb/#toc09>.
2. **Как снимать логи с устройств на Android и iOS: разбираемся с инструментами**
<https://habr.com/ru/companies/redmadrobot/articles/687184/>
3. **Просмотр журналов с помощью Logcat**
4. **View logs with Logcat** [https://developer-android-com.translate.google/studio/debug/logcat? x tr sl=auto& x tr tl=ru& x tr hl=ru](https://developer.android-com.translate.google/studio/debug/logcat?x_tr_sl=auto&x_tr_tl=ru&x_tr_hl=ru)
5. **Автоматизация тестирования Android приложений**
6. <https://habr.com/ru/companies/intel/articles/152122/>